TRIBHUVAN UNIVERSITY
INSTITUTE OF ENGINEERING
PULCHOWK CAMPUS

A
MINOR PROJECT REPORT
ON

MEDI-ASSIST
SYMPTOM-BASED COMMON DISEASE PREDICTION SYSTEM & SKIN CONDITION CLASSIFIER

SUBMITTED BY:
RAJABABU SAH LAHERA (PUL079BCT066)
RIMESH BIR SINGH (PUL079BCT068)
SUJAL TIMALSINA (PUL079BCT085)

SUBMITTED TO:
DEPARTMENT OF ELECTRONICS & COMPUTER ENGINEERING

May, 2026

Page of Approval

TRIBHUVAN UNIVERSITY
INSTITUTE OF ENGINEERING
PULCHOWK CAMPUS
DEPARTMENT OF ELECTRONICS AND COMPUTER ENGINEERING

The undersigned certify that they have read and recommend to the Institute of Engineering for acceptance of a project report entitled **"Medi-Assist: Symptom-Based Common Disease Prediction System & Skin Condition Classifier"** submitted by **Rajababu Sah Lahera**, **Rimesh Bir Singh**, and **Sujal Timalsina** in partial fulfillment of the requirements for the Bachelor's degree in Electronics & Computer Engineering.

.....	.....
Supervisor	External Examiner
.....	**.....**
.....	Assistant Professor
Department of Electronics and Computer Engineering,	Department of Electronics and Computer Engineering,
Pulchowk Campus, IOE, TU.	Pulchowk Campus, IOE, TU.

Date of approval: .....

Copyright

Acknowledgments

We sincerely express our gratitude to **Prof. Dr. Ram Krishna Maharjan**, Head of the Department of Electronics and Computer Engineering, for his inspiring guidance and for fostering an environment that encourages innovation and research.

We also wish to acknowledge the Department of Electronics and Computer Engineering, Pulchowk Campus, for providing us with the opportunity to undertake this minor project and for offering excellent facilities and resources that supported our work.

Our heartfelt thanks go to our peers and colleagues, whose insightful discussions, valuable suggestions, and collaborative spirit have greatly enriched our understanding and contributed to the quality of this proposal.

Finally, we are deeply grateful to our families for their constant support and encouragement throughout our academic journey.

Rajababu Sah Lahera
Rimesh Bir Singh
Sujal Timalsina

Abstract

Healthcare systems worldwide face significant challenges in delivering timely and effective medical assistance due to increasing patient loads, limited infrastructure, and unequal access to specialists, particularly in developing countries such as Nepal. These limitations often lead to delayed diagnosis and treatment, highlighting the need for accessible preliminary healthcare solutions. This project presents *MediAssist*, an AI-based healthcare assistant designed to provide preliminary health guidance through symptom-based disease prediction and skin disease classification.

The system consists of two main modules. The first module performs symptom-based disease prediction using machine learning techniques, specifically Decision Tree and Random Forest classifiers, trained on a dataset containing multiple diseases and associated symptoms. The optimized Random Forest model achieves a test accuracy of 96.68%, demonstrating strong predictive capability. The second module focuses on skin disease classification using a deep learning approach based on the ConvNeXt-Tiny architecture with transfer learning. Trained on dermatological image data, the model achieves an accuracy of 84.62%, outperforming baseline architectures such as EfficientNet-B0.

The system is implemented in Python, utilizing scikit-learn for machine learning and PyTorch for deep learning. A Flask-based REST API integrates both modules, while a responsive web interface enables user interaction. The system is optimized for real-time performance, providing fast and reliable predictions.

MediAssist serves as a supportive tool for early-stage health assessment and awareness. While it provides useful preliminary insights, it is intended to complement, not replace, professional medical consultation. This work demonstrates the potential of artificial intelligence in improving healthcare accessibility and supporting digital health solutions.

Keywords: Healthcare AI, Disease Prediction, Skin Classification, Machine Learning, Deep Learning, Medical Informatics

Contents

List of Figures

List of Tables

List of Abbreviations

AI	Artificial Intelligence
API	Application Programming Interface
CNN	Convolutional Neural Network
CSV	Comma-Separated Values
CV	Cross-Validation
DL	Deep Learning
DT	Decision Tree
GELU	Gaussian Error Linear Unit
GPU	Graphics Processing Unit
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
JSON	JavaScript Object Notation
ML	Machine Learning
REST	Representational State Transfer
RF	Random Forest
RGB	Red, Green, Blue
UI	User Interface
WHO	World Health Organization

1 Introduction

1.1 Background

Healthcare systems struggle to provide timely care due to too many patients and too few doctors especially in developing countries like Nepal, where there are not enough medical specialists and rural areas have limited healthcare access. As a result, patients face long waiting times, delayed diagnoses, and untreated conditions. Low medical awareness, geographical distance from hospitals, high medical costs, and a shortage of specialists make the situation more difficult.

The advent of Artificial Intelligence (AI) and Machine Learning (ML) has opened new possibilities in healthcare. Recent advancements in deep learning, particularly in Convolutional Neural Networks (CNNs) and ensemble learning methods, have demonstrated remarkable capabilities in medical image analysis and disease prediction. These technologies offer the potential to bridge the healthcare accessibility gap by providing preliminary diagnostic support that is available 24/7, regardless of geographical location.

In this context, we propose MediAssist, a Symptom-Based Common Disease Prediction System & Skin Condition Classifier which is designed to provide accessible preliminary health guidance through two core functionalities: symptom-based disease prediction and skin disease classification from images. While MediAssist is not intended to replace professional medical consultation, it serves as a valuable first-line tool that can help individuals understand their symptoms, and recognize potential common health issues early.

1.2 Problem Statement

Even though health information is available online, many people still struggle to get clear and reliable medical guidance. Patients often do not understand their symptoms, especially when they are mild or unclear, and may ignore early warning signs. The abundance of unverified information on the internet can also lead to confusion and risky self-diagnosis, as people find it difficult to determine which sources are trustworthy.

Skin diseases present particular challenges as they are often overlooked because people think they are minor problems. Early signs may be mistaken for simple irritation, and limited access to dermatologists especially in rural areas causes delays in proper treatment. High medical costs, long waiting times for appointments, and a shortage of healthcare professionals make healthcare even harder to access.

Because of these challenges, there is a strong need for a low-cost, easy-to-use system that can provide instant basic health guidance anytime and anywhere. Such a system can help people understand their symptoms, take early action, and know when to seek professional medical care.

1.3 Objectives

The objectives of the MediAssist project are as follows:

1. To develop an AI-based symptom analysis system that can accurately predict potential diseases from user-reported symptoms.
2. To construct a deep learning model for automated skin disease classification from images that can identify common skin conditions.
3. To provide comprehensive health information including disease descriptions, causes, precautions, and severity.

1.4 Scope

The scope of this project covers the development, implementation, deployment, and evaluation of the MediAssist system with the following key aspects:

1.4.1 Inclusions

- Development of a symptom-based disease predictor using a Random Forest classifier trained on a dataset covering 41 common diseases and 233 distinct symptoms, with a Decision Tree baseline for comparative evaluation.
- Development of a skin disease classifier using ConvNeXt-Tiny architecture with ImageNet transfer learning, trained on dermatological images covering 20 skin conditions.
- Creation of a comprehensive disease information database containing descriptions, causes, precautions, and severity.
- Implementation of a web-based user interface providing intuitive symptom selection, image upload functionality, and clear presentation of predictions with confidence scores.

1.4.2 Exclusions

- This system does not provide medical diagnosis or treatment recommendations, it offers preliminary guidance only.
- Does not handle emergency medical situations or life-threatening conditions.
- Is not a replacement for doctors.
- Not intended for rare or highly specialized medical conditions outside the training dataset scope.

The project focused on providing accessible preliminary health guidance while clearly maintaining the boundary between AI assistance and professional medical care.

2 Literature Review

Over the past decade, machine learning and deep learning have been widely applied to medical diagnosis. This chapter highlights key studies in symptom-based disease prediction and medical image classification that informed our approach in developing MediAssist.

2.1 Related Work

2.1.1 Symptom-Based Disease Prediction

Machine learning has been widely applied to symptom-based disease prediction. Kolukisa et al. (2018) compared classifiers like Decision Trees, SVM, Naïve Bayes, and Random Forest, finding Random Forest achieved the best accuracy (85–92%) due to its ability to handle high-dimensional symptom data and avoid overfitting [1]. Uddin et al. (2019) used a dataset of 4,000+ patient records and 132 symptoms, reporting 95% accuracy with Random Forest while addressing class imbalance [2]. Chen et al. (2020) combined expert systems with machine learning in a two-stage approach to improve both accuracy and interpretability [3]. Sharma et al. (2021) noted that deep learning models need larger datasets and more resources, making ensemble methods like Random Forest more practical for limited tabular symptom data [4].

2.1.2 Deep Learning in Medical Image Analysis

Deep learning has transformed medical image analysis, especially in dermatology. Esteva et al. (2017) showed that a CNN (Inception-v3) could classify skin cancer with accuracy comparable to 21 dermatologists using 129,450 clinical images [5]. Haenssle et al. (2018) found CNNs outperformed average dermatologists in melanoma detection on 12,000 dermoscopic images, highlighting the importance of diverse, high-quality training data [6]. Tschandl et al. (2018) created the HAM10000 dataset, a benchmark of dermatoscopic images across seven disease categories, enabling fair model comparisons [7]. Liu et al. (2022) introduced ConvNeXt, a modern, efficient CNN architecture with competitive performance and lower computational cost, suitable for medical imaging [8]. Tan and Le (2019) introduced EfficientNet, a family of scalable CNNs based on compound coefficient scaling, which achieves strong accuracy with fewer parameters, making it suitable as a baseline in resource-constrained settings [9].

2.1.3 Gaps in Existing Solutions

A review of existing literature and digital health systems highlights several limitations that MediAssist aims to address:

- **Limited Integration:** Many existing systems primarily focus either on symptom-based disease prediction or image-based diagnosis, rather than providing a tightly integrated multi-modal solution within a single platform.

- **Accessibility Barriers:** Several existing platforms may require user registration, subscription access, or have limited availability in certain regions, which can restrict their usability for under-served populations.
- **Insufficient Contextual Information:** While most systems provide predictions, many offer limited explanations regarding disease descriptions, causes, and precautionary measures, reducing their educational value for users.

MediAssist addresses these limitations by providing an integrated, freely accessible system that combines symptom-based and image-based diagnosis along with structured disease information and performance evaluation.

2.1.4 Comparison with Existing Digital Health Tools

Table 2.1 compares MediAssist with popular AI-based health assistants.

Table 2.1: Feature comparison with existing systems

Feature	**MediAssist**	**WebMD**	**Ada Health**	**Google Health (Tools)**
Symptom-based prediction	Yes	Yes	Yes	Yes
Skin image classification	Yes	No	No	Yes (dermatology)
Free access	Yes	Yes	Yes	Mostly free
Registration required	No	Partial	Yes	Yes
Open source	Yes	No	No	No
Runs offline (possible)	No	No	No	No
Trained on public datasets	Yes	Proprietary	Proprietary	Proprietary

Unlike commercial systems, MediAssist is completely free, open-source, and does not require user accounts. However, it offers fewer diseases and lower clinical validation compared to established products. Additionally, existing systems generally provide broader medical coverage and are trained on clinically validated datasets, resulting in higher reliability. However, they rely on proprietary models, often require user registration, and depend on continuous internet connectivity, which may limit accessibility and transparency.

2.2 Related Theory

2.2.1 Decision Tree Classifier

The Decision Tree is a supervised learning algorithm that constructs a tree-like model of decisions by recursively splitting the dataset based on feature values. At each node, the algorithm selects the feature that maximizes information gain or minimizes impurity (commonly measured using Gini index or entropy), resulting in progressively purer subsets of data.

For classification tasks, the model predicts the class label by traversing from the root node to a leaf node, following decision rules based on input features. The final prediction corresponds to the majority class at the leaf node. In this project, the Decision Tree is configured to grow fully (i.e., without depth restriction), allowing it to capture complex patterns in the training data.

While Decision Trees are simple and interpretable, they are prone to overfitting, especially when fully grown. However, they serve as an important baseline model for comparison with ensemble methods such as Random Forest.

Key advantages include: (1) ease of interpretation and visualization, (2) ability to handle both numerical and categorical data, (3) minimal data preprocessing requirements, and (4) fast training and prediction. Despite their limitations, Decision Trees provide valuable insights into feature importance and decision-making processes in medical diagnosis tasks.

2.2.2 Random Forest Classifier

Random Forest, proposed by Breiman (2001), is an ensemble learning method that operates by constructing multiple decision trees during training and outputting the mode of the classes for classification tasks [10]. The algorithm's strength lies in its ability to reduce overfitting through bootstrap aggregation (bagging) and random feature selection at each split, making the trees less correlated and the ensemble more robust.

For classification, each tree in the forest votes for a class, and the forest outputs the class with the majority vote. The probability estimate for each class is calculated as the proportion of trees voting for that class.

Key advantages for medical applications include: (1) ability to handle high-dimensional data with many features relative to samples, (2) built-in feature importance ranking that provides interpretability, (3) robust performance with default hyperparameters, reducing the need for extensive tuning, (4) capability to handle missing data through proximity-based imputation, and (5) resistance to overfitting through ensemble averaging.

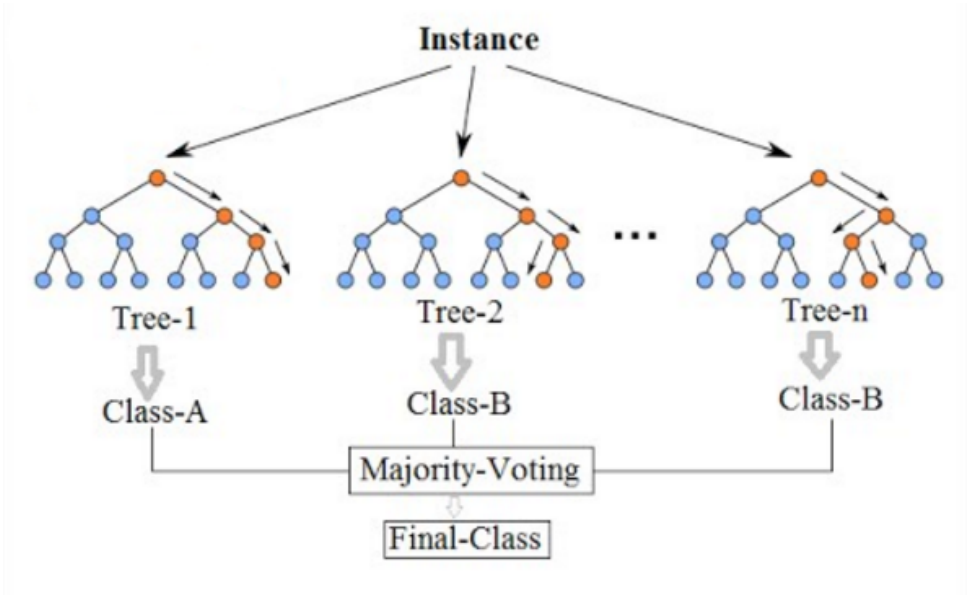

Figure 2.1: Random Forest Classifier

The process of classifying an Instance follows these steps:

1. **Input Instance:** The process begins with a single data point or instance x that needs to be classified.

2. **Parallel Processing (The Forest):** The instance is fed simultaneously into n independent decision trees ($Tree_1, Tree_2, \ldots, Tree_n$).

3. **Individual Predictions:** Each tree T_i produces its own result:

   - $T_1(x) \rightarrow$ Class-A
   - $T_2(x) \rightarrow$ Class-B
   - $T_n(x) \rightarrow$ Class-B

4. **Majority Voting:** The model collects all predictions and performs a tally. This "Wisdom of the Crowd" approach helps mitigate the errors of any single biased tree.

5. **Final Output:** The class with the highest frequency becomes the Final Class (in this case, Class-B).

Mathematical Formulation: Let the ensemble of classifiers be denoted as: $H = \{h_1(x), h_2(x), \ldots, h_n(x)\}$. The final classification $\hat{y}$ for an input x is determined by:

$$\hat{y} = \arg \max_{j \in \{1,\ldots,C\}} \sum_{i=1}^{n} I(h_i(x) = j) \tag{2.1}$$

Where:

- n is the number of trees in the forest.
- $I(\cdot)$ is the indicator function (1 if true, 0 otherwise).
- j represents the available class labels.

2.2.3 Convolutional Neural Networks

Convolutional Neural Networks (CNNs), popularized by LeCun et al. (1998) and revolutionized by Krizhevsky et al. (2012) with AlexNet [11,12], are specialized neural networks designed to process grid-like data such as images.

CNNs consist of three main types of layers: convolutional layers that apply learnable filters to input images through convolution operations, extracting features like edges, textures, and patterns; pooling layers that perform downsampling to reduce spatial dimensions, decreasing computational requirements and providing translation invariance; and fully connected layers that connect neurons from the previous layer to every neuron in the current layer, typically used in the final stages for classification based on extracted features.

2.2.4 EfficientNet Architecture

EfficientNet, proposed by Tan and Le (2019), is a family of convolutional neural networks that improves performance by scaling the model in a balanced way [9]. Instead of increasing only depth or width, it scales depth, width, and input resolution together, which leads to better accuracy while keeping the model efficient.

EfficientNet-B0 is the base version of the architecture. It is built using mobile inverted bottleneck convolution (MBConv) blocks, which use depthwise separable convolutions and squeeze-and-excitation modules to extract features efficiently.

In this project, EfficientNet-B0 is used as a baseline model for skin disease classification. It performs well even with limited data and is suitable for transfer learning.

The main advantages of EfficientNet include: (1) high accuracy with fewer parameters, (2) efficient scaling of the model, and (3) good performance on devices with limited computational resources.

2.2.5 ConvNeXt Architecture

Liu et al. (2022) introduced ConvNeXt as an improved convolutional neural network designed to perform as well as Vision Transformers [8]. The model updates traditional CNN designs using several modern techniques. It follows a stage-based structure similar to transformers, where layers are grouped in stages (for ConvNeXt-Tiny, the number of blocks in each stage is 3–3–9–3).

It uses depthwise separable convolutions, which reduce the number of parameters and computation while still maintaining good performance. Larger convolution kernels (7×7 instead of the usual 3×3) are used to capture more information from the image, similar to how attention mechanisms work.

The architecture also uses an inverted bottleneck design, where features are first expanded and then compressed to improve representation. Additionally, it replaces Batch Normalization and ReLU with Layer Normalization and GELU activation, which helps make training more stable and effective.

The main advantages of ConvNeXt include: (1) high accuracy comparable to transformer-based models, (2) improved feature extraction using larger convolution kernels, and (3) stable and effective training due to the use of Layer Normalization and GELU activation.

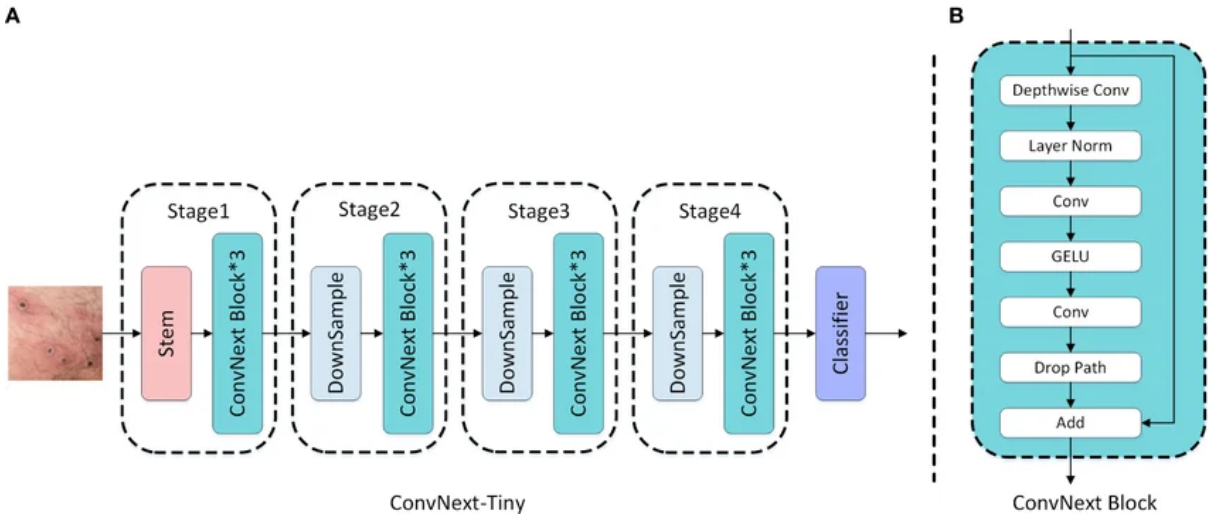

Figure 2.2: A) ConvNext-Tiny Architecture B) ConvNext Block Structure

2.2.5.1 Overall Architecture

The architecture follows a hierarchical structure consisting of a stem, multiple stages, and a classifier:

$$\text{Input Image} \rightarrow \text{Stem} \rightarrow \text{Stage 1} \rightarrow \text{Stage 2} \rightarrow \text{Stage 3} \rightarrow \text{Stage 4} \rightarrow \text{Classifier}$$

- **Stem:** The stem is the initial layer that processes the input image and converts it into feature maps using convolution operations.
- **Stages:** The network is divided into four stages. Each stage consists of multiple ConvNeXt blocks. Downsampling is applied between stages to reduce spatial resolution and increase feature depth.
  - **Stage 1:** Captures low-level features such as edges and textures.
  - **Stage 2:** Learns intermediate features like shapes.
  - **Stage 3:** Extracts higher-level patterns and object parts.
  - **Stage 4:** Learns semantic and global representations.
- **Downsampling:** Reduces the spatial dimensions of feature maps while increasing the number of channels, enabling hierarchical feature learning.
- **Classifier:** A fully connected layer that produces the final class prediction.

2.2.5.2 ConvNeXt Block

The ConvNeXt block is the fundamental building unit of the architecture. It improves upon traditional convolutional blocks using modern design choices.

Each block consists of the following components:

1. **Depthwise Convolution:** Applies convolution independently to each channel, capturing spatial information efficiently.
2. **Layer Normalization:** Normalizes features to stabilize training.
3. **Pointwise Convolution (1×1):** Mixes information across channels.
4. **GELU Activation:** A smooth non-linear activation function that improves learning.
5. **Convolution Layer:** Further refines extracted features.
6. **Drop Path:** A regularization technique that randomly drops paths during training to reduce overfitting.
7. **Residual Connection:** Adds the input to the output of the block to preserve information and improve gradient flow.

2.2.6 Transfer Learning

Transfer learning leverages knowledge gained from solving one task to improve performance on a related task [13]. In medical imaging, this typically involves pre-training on large general datasets like ImageNet (1.2 million images across 1,000 classes), then fine-tuning on smaller medical datasets.

The effectiveness stems from the hierarchical nature of feature learning in CNNs, where lower layers learn general features (edges, textures, color gradients) broadly applicable across domains, while higher layers learn task-specific features. By preserving general features from pre-training and adapting only task-specific layers, transfer learning enables high performance even with limited medical training data.

2.2.7 Evaluation Metrics for Classification

For multi-class classification tasks in medical applications, several metrics are essential beyond simple accuracy. Precision (Positive Predictive Value) measures the proportion of predicted positives that are truly positive, indicating prediction reliability. Recall (Sensitivity or True Positive Rate) measures the proportion of actual positives correctly identified, indicating the model's ability to find all positive instances. F1-Score provides the harmonic mean of precision and recall, balancing both concerns particularly important in medical applications where both false positives (unnecessary worry) and false negatives (missed diagnoses) have significant consequences. The Confusion Matrix provides detailed performance across all classes, revealing which classes are commonly confused and identifying systematic errors for improvement.

3 Methodology

This chapter details the systematic approach employed in developing MediAssist, from initial feasibility assessment through implementation and deployment.

3.1 Feasibility Study

Before committing to full-scale development, we conducted a comprehensive feasibility analysis to ensure project viability.

3.1.1 Technical Feasibility

We evaluated whether the required technologies, algorithms, and infrastructure were available and suitable for our objectives. Our investigation confirmed that Decision Tree and Random Forest classifiers are well-established and readily available through scikit-learn with extensive documentation. ConvNeXt-Tiny and EfficientNet-B0 models with pre-trained ImageNet weights are available through PyTorch's torchvision library. Symptom-disease datasets are publicly available on Kaggle, and dermatological image datasets provide sufficient labeled images for both models.

For computational resources, while deep learning training typically requires GPUs, we had access to Google Colab providing free GPU resources adequate for training both ConvNeXt-Tiny and EfficientNet-B0. Model inference for both modules runs efficiently on standard CPU hardware, making deployment feasible without expensive infrastructure. All required software tools (Python, PyTorch, scikit-learn, Flask) are open-source and well-documented. Based on these findings, we concluded that the project was technically feasible with available resources.

3.1.2 Economic Feasibility

Development costs were minimal: zero cost for all software tools using open-source libraries, free dataset acquisition through Kaggle, and free computational resources through Google Colab for training. For deployment, we utilized Hugging Face Spaces which offers free hosting for machine learning applications, eliminating deployment costs entirely making it highly economically feasible for a student project.

3.2 Requirement Analysis

We conducted a systematic analysis to identify the functional and non-functional requirements for MediAssist.

3.2.1 Functional Requirements

The system must provide the following functional capabilities:

1. **Symptom Input and Selection**

- Provide a comprehensive list of 233 unique symptoms organized into clearly labeled categories to help users navigate and select symptoms
- Enable multi-select checkbox interface for symptom selection
- Include search functionality to quickly locate specific symptoms
- Display user-friendly error message when no symptoms are selected before submission

2. **Disease Prediction from Symptoms**
    - Accept selected symptoms as input and encode them into a 233-dimensional binary feature vector
    - Process symptoms through the trained Random Forest model
    - Return top 3 predicted diseases with normalized probability scores
3. **Image Upload and Processing**
    - Accept image uploads in common formats (JPEG, PNG)
    - Display image preview before submission
    - Display user-friendly error message when an unsupported image format is uploaded
4. **Skin Disease Classification**
    - Preprocess uploaded images (resize to 224×224, normalize with ImageNet statistics)
    - Classify images using the trained ConvNeXt-Tiny model
    - Return predicted skin condition with confidence percentage
5. **Disease Information Presentation**
    - Display comprehensive information for predicted conditions
    - Include: disease description, common causes, severity, precautions

3.2.2 Non-Functional Requirements

Beyond specific functions, the system must meet quality standards across several dimensions:

1. **Performance**
    - Symptom prediction response time: <1 second
    - Image classification response time: <3 seconds
    - Support at least 5 concurrent users without degradation
2. **Accuracy**
    - Symptom-based prediction: minimum 85% accuracy on test set
    - Skin disease classification: minimum 80% accuracy on test set

3. **Usability**

   - Intuitive interface requiring no training or instructions
   - Accessible to users with varying technical literacy
   - Responsive design functioning on desktop, tablet, and mobile

3.3 Data Collection

High-quality, representative data is essential for building accurate machine learning models. This section describes the datasets used for both modules and the reasoning behind their selection.

3.3.1 Module 1: Symptom-Disease Dataset

For symptom-based disease prediction, the dataset was sourced from Kaggle as *Disease Symptom Prediction Dataset* (itachi9604/disease-symptom-description-dataset) and is licensed under CC0 Public Domain, allowing free academic use.

The dataset is structured in a tabular CSV format, where each row corresponds to a single patient record. It contains one target column (`Disease`) and seventeen symptom columns (`Symptom_1` through `Symptom_17`), each representing a symptom present in that patient. Columns without a symptom are left empty, allowing a flexible number of symptoms per record.

Initially, the dataset contained 54,920 rows. Duplicate records were removed, resulting in 16,857 unique entries spanning 41 disease classes and 233 distinct symptoms. The removed rows had same disease label and identical sets of symptoms which added no additional clinical information.

The distribution of records across disease classes is imbalanced, ranging from 22 (acne) to 911 (hypothyroidism). To mitigate potential bias toward more common diseases, the model training applied class weighting, giving higher influence to less frequent classes. This ensures the model learns to recognize minority classes effectively.

Table 3.1: Sample of the Symptom-Disease Dataset

Disease	**Symptom_1**	**Symptom_2**	**Symptom_3**	**Symptom_4**	**...**	**Symptom_17**
Fungal Infection	itching	skin_rash	nodal_skin_eruptions		...	
Diabetes	frequent_urination	fatigue	blurred_vision		...	
Common Cold	cough	fever	sore_throat	runny_nose	...	
Dengue	fever	headache	joint_pain	rash	...	
Typhoid	fever	abdominal_pain	weakness	diarrhea	...	

3.3.2 Module 2: Skin Disease Image Dataset

We used the Skin Disease Dataset from Kaggle (`pacificrm/skindiseasedataset`), organized into train/test split directories and is licensed under CC0 Public Domain, allowing free academic use. The dataset consists of 13,000+ images across 20 skin disease categories including Acne, Actinic Keratosis, Bullous, Candidiasis, Drug Eruption, Infestations/Bites, Lichen, Lupus, Moles, Psoriasis, Rosacea, Seborrheic Keratoses, Skin Cancer, Sun/Sunlight Damage, Tinea, Unknown, Vascular Tumors, Vasculitis, Vitiligo, and Warts.

3.4 Data Preprocessing

3.4.1 Module 1: Symptom-Disease Dataset

The dataset was first cleaned by removing duplicate records and standardizing all disease and symptom names to ensure consistency. Missing values were preserved to accurately represent unreported symptoms.

Each patient record was then transformed into a fixed-length binary feature vector using multi-hot encoding, where each position corresponds to a unique symptom and indicates its presence or absence. This representation allows multiple symptoms to be captured simultaneously in a structured format suitable for machine learning models.

Finally, the dataset was divided into training and test sets using an 80:20 split. Stratified sampling was applied to maintain the distribution of all disease classes across both sets, ensuring balanced and reliable model evaluation.

3.4.2 Module 2: Skin Disease Image Dataset

The skin disease image dataset was organized into training and testing sets, covering 20 distinct skin conditions. Images were prepared in a structured format suitable for deep learning models.

To improve model performance and generalization, training images were augmented using transformations such as resizing, cropping, flipping, and color adjustments. All images were then normalized to a standard scale to ensure consistency during training.

For evaluation, validation images were resized and normalized in the same manner but without augmentation, ensuring that model performance is assessed on unaltered data.

3.5 System Design and Implementation

3.5.1 System Architecture

We implemented a three-tier architecture: Presentation Tier (Frontend) using HTML, CSS, and JavaScript for user interface handling interactions and display; Application Tier (Backend) using Flask REST API serving as intermediary between frontend and models, managing routing and responses; and Data Tier consisting of trained models (Random Forest, ConvNeXt-Tiny) and disease information stored in CSV files.

3.5.2 Module 1: Symptom Predictor Implementation

Baseline Model - Decision Tree: A Decision Tree classifier was first trained as a baseline model to evaluate the performance of a single-tree approach. This provided a reference point for comparing the improvements achieved by ensemble methods.

Random Forest Hyperparameter Tuning: A Random Forest classifier was then used and optimized using grid search with cross-validation over key hyperparameters such as the number of trees, tree depth, and minimum samples per leaf. This process identified an optimal configuration that significantly improved performance over the baseline model. Visualization techniques such as heatmaps were used to analyze the effect of different parameter combinations on model accuracy.

3.5.3 Module 2: Skin Classifier Implementation

Baseline Model - EfficientNet-B0: EfficientNet-B0 was trained as a baseline using ImageNet pre-trained weights to establish a reference performance for the skin classification task.

ConvNeXt-Tiny Model: The ConvNeXt-Tiny model was initialized with ImageNet pre-trained weights and fine-tuned end-to-end using the AdamW optimizer and a Cosine Annealing learning rate scheduler.

Model Selection: ConvNeXt-Tiny was selected as the final skin classifier as it demonstrated superior performance compared to EfficientNet-B0 under comparable training conditions.

3.5.4 Backend Implementation

A Flask-based REST API was developed to serve the machine learning models. The system includes three main endpoints: a disease prediction endpoint that accepts a list of symptoms and returns the top three predicted diseases with probability scores, a skin disease classification endpoint that processes uploaded images and returns the predicted condition with confidence, and a utility endpoint that provides the list of all available symptoms for frontend integration.

3.5.5 Frontend Implementation

The frontend was designed as a web-based interface to interact with both modules of the system. It includes a symptom-based prediction interface with searchable and selectable symptoms, as well as a skin disease classification interface that allows users to upload images for analysis.

The system provides structured result displays showing the top predictions with confidence scores and includes a disclaimer indicating that the outputs are intended for informational purposes only and not as a medical diagnosis.

3.6 Testing

To ensure the reliability and correctness of MediAssist, testing was performed across multiple levels covering individual components, API endpoints, and overall system behavior.

3.6.1 Unit Testing

Unit testing was performed on individual components of the system to verify that each module functioned correctly in isolation. This included validating the multi-hot encoding function to ensure that symptom inputs were correctly converted into 233-dimensional binary vectors.

The image preprocessing pipeline was tested to ensure that uploaded images of varying sizes and formats were correctly resized to 224×224, normalized using ImageNet statistics, and converted into PyTorch tensor format without errors.

3.6.2 API Integration Testing

REST API endpoints were tested using the Thunder Client extension in Visual Studio Code to verify correct end-to-end communication between the frontend and backend.

The `/predict_symptom` endpoint was tested using POST requests with different symptom combinations, including single-symptom inputs, and empty inputs. The system correctly returned the top-3 predicted diseases with associated probability scores and handled invalid inputs with appropriate error messages.

The `/predict_skin` endpoint was tested by uploading images in JPEG and PNG formats using multipart form-data requests within Thunder Client. Images of varying resolutions and quality levels were used to evaluate robustness. The model successfully returned predicted skin conditions along with confidence scores in JSON format.

The `/get_all_symptoms` endpoint was validated to confirm that the complete list of 233 symptoms was correctly returned for frontend integration.

3.6.3 Performance Testing

Performance testing was conducted to evaluate system responsiveness under normal usage conditions. Symptom-based predictions were consistently returned in under 1 second, while skin disease classification responses were generated in under 3 seconds. Response times were measured manually using browser-based testing.

3.6.4 User Testing

The deployed application on Hugging Face Spaces was evaluated through informal user testing with a small group of peers. Users interacted with both modules by selecting symptoms and uploading skin images.

Feedback was collected qualitatively and indicated that the symptom search and multi-select interface improved usability, while confidence scores and ranked predictions enhanced interpretability. Users also found the disease descriptions and precautionary information helpful for understanding the results.

Based on the feedback, minor improvements were made to UI labeling and result presentation before final submission.

3.7 Deployment

We deployed MediAssist on Hugging Face Spaces, a free platform for hosting machine learning applications. The deployed application is accessible at:
`https://huggingface.co/spaces/rimesh-061/MediAssist`

4 Experimental Setup

This chapter describes the detailed experimental procedures followed to develop, train, and evaluate both modules of MediAssist.

4.1 Development Environment

4.1.1 Hardware Resources

Development was conducted on team members' personal laptops utilizing Google Colab for training our models.

4.1.2 Software Environment

- **Operating System:** Windows 10
- **Programming Language:** Python 3.9.7
- **Libraries and Frameworks:**
  - **scikit-learn 1.0.2** – Decision Tree and Random Forest classifiers, train-test split, grid search with cross-validation
  - **joblib** – Saving and loading the trained Random Forest model
  - **PyTorch 1.12.1** – Deep learning framework for ConvNeXt-Tiny and EfficientNet-B0
  - **torchvision 0.13.1** – Pre-trained models, image preprocessing, data augmentation, ImageNet normalization
  - **Flask 2.2.2** – REST API development, request routing, model serving
  - **pandas 1.4.3** – Data loading and cleaning (CSV parsing, text normalization, deduplication), multi-hot feature matrix creation
  - **numpy 1.22.4** – Class weight index generation, tensor-to-array conversion for sklearn metrics
  - **Pillow 9.2.0** – Image loading and basic processing
  - **Matplotlib 3.5.2** – Visualization (confusion matrices, loss/accuracy curves, heatmaps)
  - **HTML, CSS, JavaScript** – Frontend user interface (symptom selection, image upload, result display)
- **Development Tools:**
  - **Google Colab** – Free GPU-accelerated training for both machine learning and deep learning models
  - **VS Code 1.70** – Primary code editor
  - **Hugging Face Spaces** – Free deployment platform for the complete web application
  - **Kaggle** – Source of datasets (symptom-disease CSV, skin disease images)

4.2 Module 1: Symptom-Based Disease Predictor Experiments

4.2.1 Data Preprocessing

The dataset initially contained 54,920 patient records with 18 columns (one disease column and 17 symptom columns). After preprocessing, the dataset was reduced to 16,857 unique records.

In addition to removing exact duplicates, order-insensitive duplicate records were also eliminated. This ensures that records containing the same set of symptoms in different column orders are treated as duplicates. For example, the records shown below contain identical symptoms but in different orders and are therefore considered duplicates:

Table 4.1: Example of Order-Insensitive Duplicate Records

Disease	**Symptom_1**	**Symptom_2**	**Symptom_3**
Fungal infection	itching	skin_rash	nodal_skin_eruptions
Fungal infection	skin_rash	nodal_skin_eruptions	itching

To ensure consistency, all disease and symptom names were standardized by converting text to lowercase and removing leading and trailing spaces. This ensured that variations such as "Fever", " fever", and "FEVER" were treated as the same symptom.

After cleaning, the dataset contains 41 disease classes and 233 unique symptoms. No completely empty records were found in the dataset.

4.2.2 Multi-Hot Encoding of Symptoms (Feature Engineering Process)

Each patient record was converted into a fixed-length binary vector of 233 positions, one for each unique symptom in the dataset. A value of 1 indicates that a symptom is present in the record, while 0 indicates that it is absent. This type of representation is called *multi-hot encoding* because multiple symptoms can be present at the same time.

Table 4.2 illustrates the multi-hot encoding for four hypothetical patient records across a representative subset of five symptoms drawn from the vocabulary.

Table 4.2: Example of Multi-Hot Symptom Encoding (5 symptoms shown out of 233)

Record	**fever**	**cough**	**fatigue**	**headache**	**skin_rash**	**Disease Label**
Patient 1	1	1	0	0	0	Common Cold
Patient 2	1	0	1	1	0	Typhoid
Patient 3	0	0	1	0	1	Fungal Infection
Patient 4	1	0	1	0	1	Dengue

1 = symptom present 0 = symptom absent

In the actual dataset all 233 symptom columns are present. Each patient record maps to a row of 233 binary values, where only the positions corresponding to that patient's

reported symptoms are set to 1 and all remaining positions are 0. The vector is sparse in practice: a typical record has between 3 and 17 non-zero entries out of 233 positions, since each row contains at most 17 symptom slots (`Symptom_1` through `Symptom_17`).

4.2.3 Train-Test Split

The encoded dataset was divided into training and test sets, with 80% of the records used for training and 20% reserved for testing. Stratified splitting was applied to ensure that all 41 disease classes appear in roughly the same proportions in both sets. This prevents any disease class from being underrepresented in the training or test data.

4.2.4 Baseline Model: Decision Tree

A single `DecisionTreeClassifier` was used as the baseline model to provide a reference performance before applying ensemble learning methods. The model was configured as follows:

- `random_state=42`: Ensures reproducibility of results across runs
- `class_weight="balanced"`: Handles class imbalance by assigning higher weights to under-represented disease classes
- `max_depth=None`: Allows the tree to grow fully until all leaves contain fewer than the minimum samples
- `min_samples_leaf=1`: Ensures that each leaf node contains at least one training sample

The model was trained using the full training dataset and evaluated on the test set without any hyperparameter tuning. This configuration represents a high-variance, fully grown decision tree and serves as a baseline for comparison with ensemble-based methods such as Random Forest.

4.2.5 Random Forest with Hyperparameter Tuning

A `RandomForestClassifier` was used as the main ensemble model to improve prediction performance over the baseline Decision Tree. The model was first initialized with default ensemble settings and class balancing enabled:

- `random_state=42`: Ensures reproducibility of results
- `class_weight="balanced"`: Handles class imbalance by assigning higher importance to minority disease classes
- `n_jobs=-1`: Enables parallel computation using all available CPU cores for faster training

4.2.5.1 Hyperparameter Optimization

To obtain the optimal configuration, Grid Search with 5-fold cross-validation (`cv=5`) was applied. The model was evaluated over multiple combinations of hyperparameters:

- `n_estimators`: {50, 100, 200} (number of decision trees in the forest)
- `max_depth`: {5, 10, 15, 20} (maximum depth of each tree)
- `min_samples_leaf`: {1, 2, 3, 4} (minimum samples required at a leaf node)

The Grid Search evaluates all combinations of these parameters using cross-validation and selects the configuration that maximizes classification accuracy.

4.2.5.2 Hyperparameter Analysis

To visualize the effect of hyperparameter combinations on model performance, 3 heatmaps were generated that show pairwise relationships between key hyperparameters:

- `max_depth` vs `n_estimators`
- `min_samples_leaf` vs `n_estimators`
- `max_depth` vs `min_samples_leaf`

These visualizations help identify how model complexity (tree depth and number of estimators) and regularization (minimum samples per leaf) influence classification accuracy.

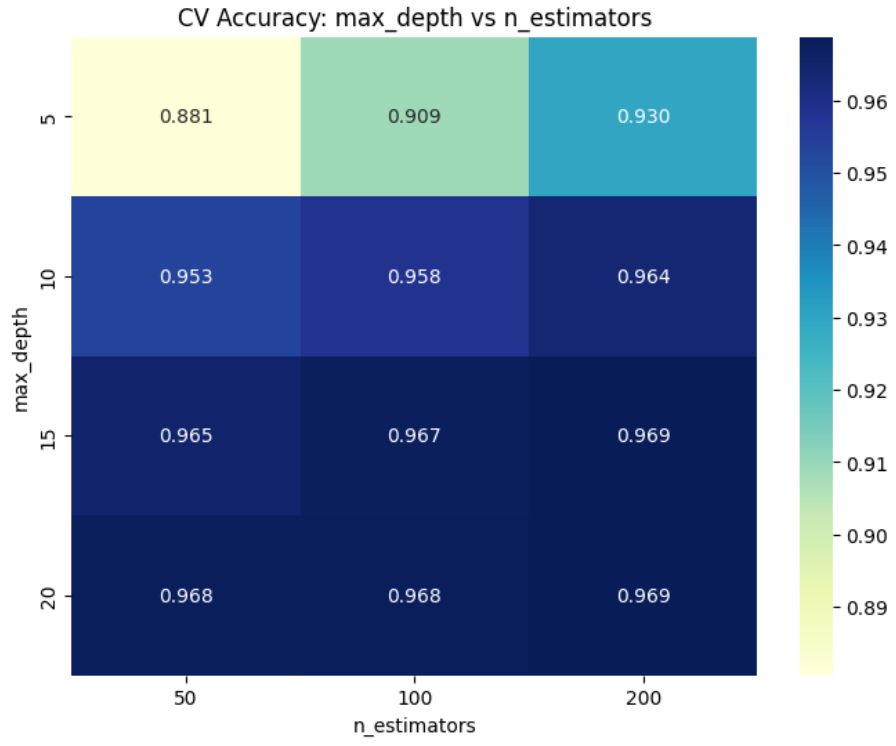

Figure 4.1: Cross-validation accuracy: max_depth vs n_estimators

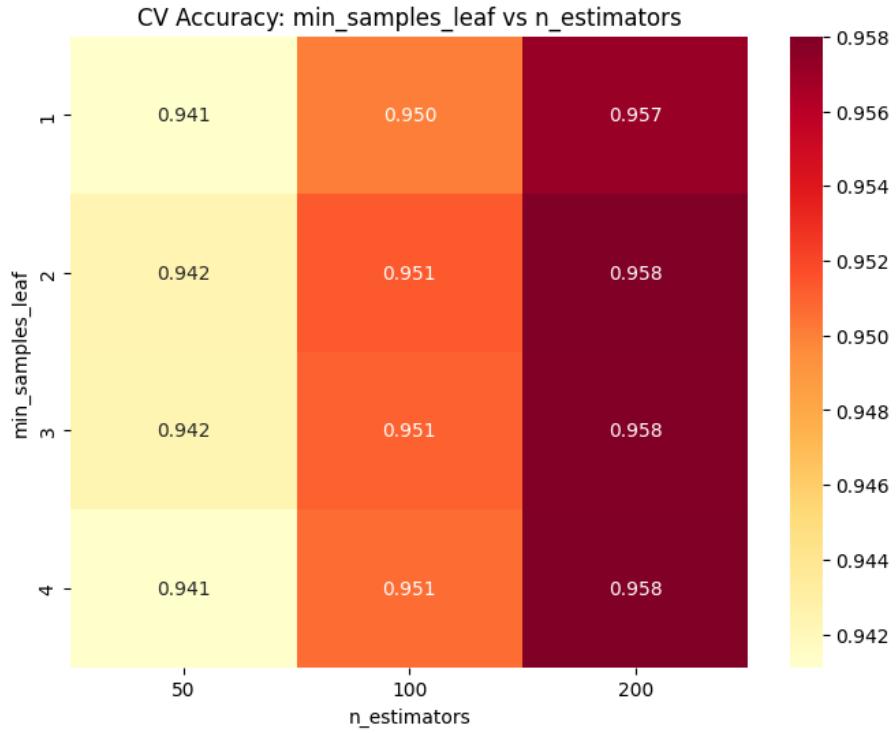

Figure 4.2: Cross-validation accuracy: min_samples_leaf vs n_estimators

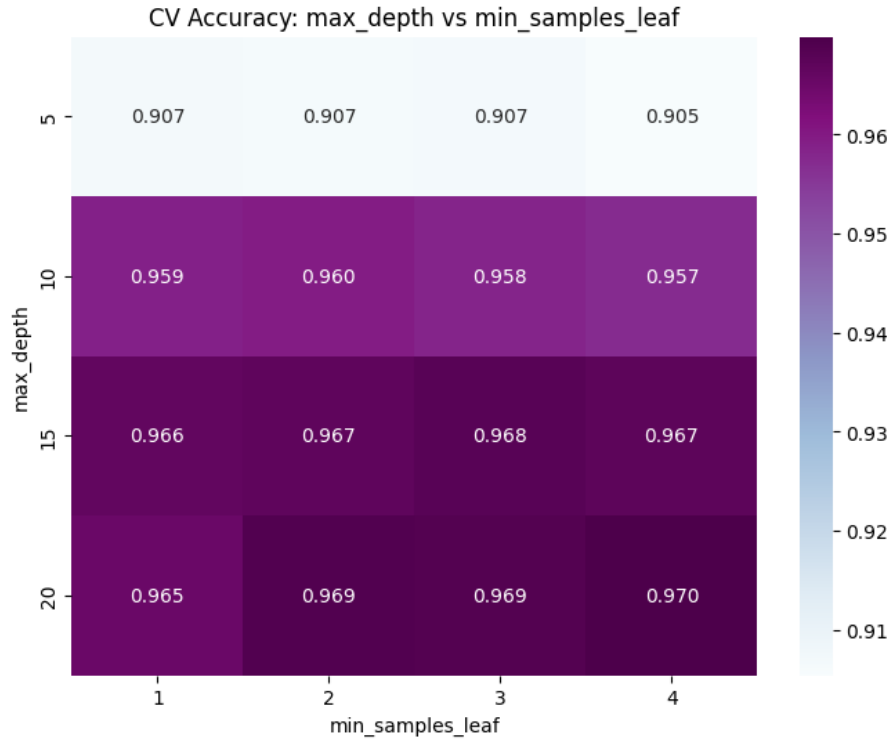

Figure 4.3: Cross-validation accuracy: max_depth vs min_samples_leaf

4.2.6 Model Selection and Evaluation

The best model from Grid Search (`best_estimator_`) was selected as the final Random Forest classifier. It was then evaluated on the unseen test set to measure generalization performance. The use of cross-validation ensures that the selected model is robust and not overfitted to a single train-test split.

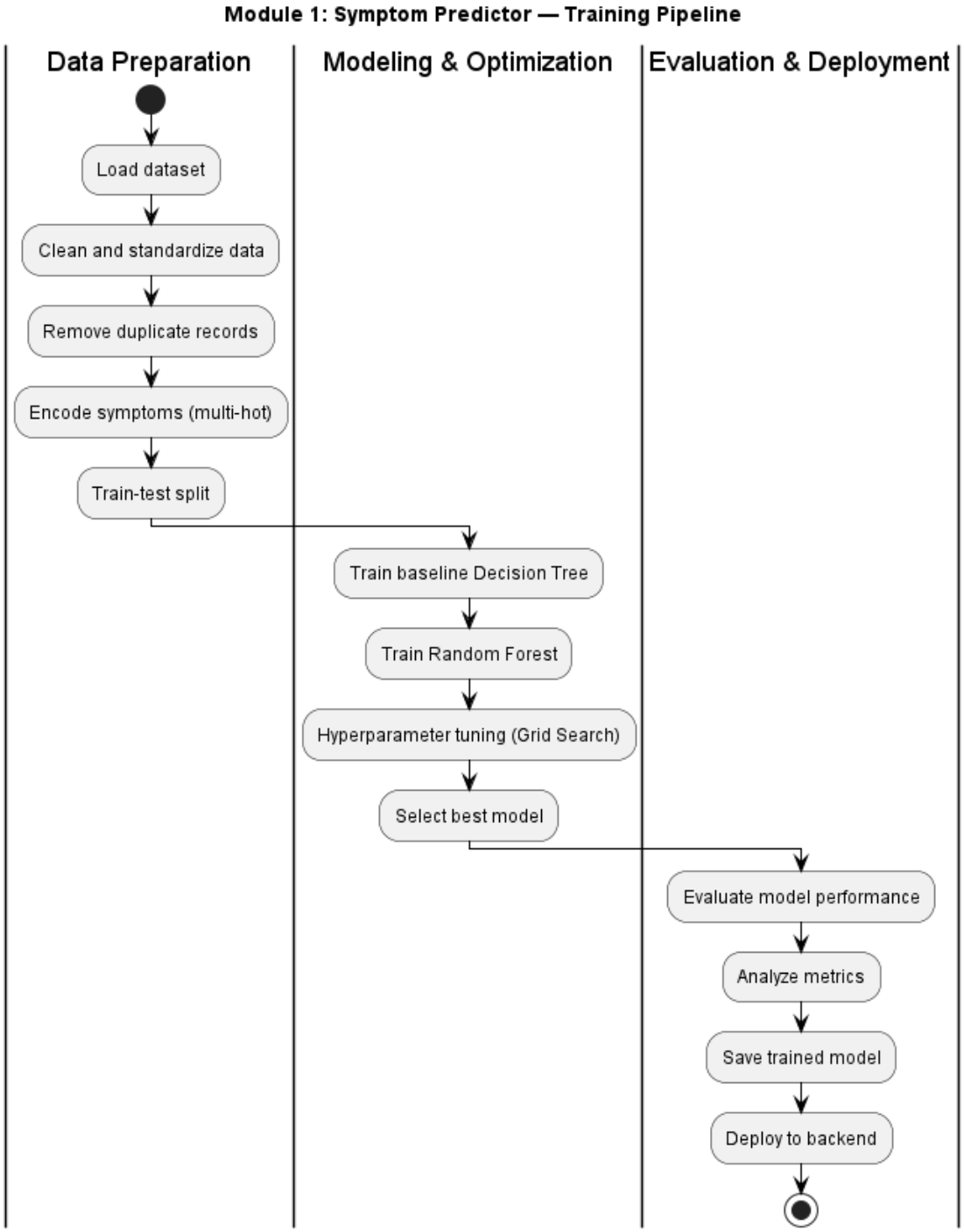

Figure 4.4: Module 1: Symptom Predictor - Training Pipeline

4.3 Module 2: Skin Disease Classifier Experiments

4.3.1 Image Preprocessing Pipeline

A unified preprocessing pipeline was used for both ConvNeXt-Tiny and EfficientNet-B0 to ensure fair comparison under identical input conditions. All images were resized to a fixed resolution of 224×224 and normalized using standard ImageNet statistics.

4.3.1.1 Training Preprocessing

During training, data augmentation was applied to improve generalization and reduce overfitting. The augmentation pipeline included random resized cropping with scale $(0.7, 1.0)$ and aspect ratio $(0.9, 1.1)$, random horizontal flipping with probability 0.5, and color jittering (brightness, contrast, saturation, and hue variations).

After augmentation, images were converted into tensor format and normalized using ImageNet mean $[0.485, 0.456, 0.406]$ and standard deviation $[0.229, 0.224, 0.225]$.

4.3.1.2 Validation Preprocessing

For validation sets, no augmentation was applied to ensure unbiased evaluation. Images were only resized to 224×224, converted to tensors, and normalized using the same ImageNet statistics as the training set.

4.3.2 Baseline Model: EfficientNet-B0

The EfficientNet-B0 model was used as the baseline deep learning architecture for skin disease classification. The model was initialized using ImageNet pre-trained weights and modified for the target number of classes.

4.3.2.1 Model Setup

The network was instantiated using a pre-trained EfficientNet-B0 backbone, where the final classification layer was replaced to match the number of skin disease classes:

- `EfficientNet-B0` initialized with ImageNet pre-trained weights
- Final fully connected layer adapted to `len(train_ds.classes)` output classes

4.3.2.2 Loss Function and Optimization

The training objective and optimization strategy were defined as follows:

- **Loss Function:** Cross-Entropy Loss with class weighting to handle dataset imbalance
- **Optimizer:** AdamW optimizer with learning rate 1×10^{-4}
- **Scheduler:** Cosine Annealing Learning Rate Scheduler with $T_{max} = 30$ epochs

4.3.2.3 Training Procedure

The model was trained for 30 epochs using mini-batch gradient descent. In each iteration, forward propagation was performed to compute predictions, and the loss was calculated using Cross-Entropy Loss. Backpropagation was then used to compute gradients, and model parameters were updated using the AdamW optimizer.

Training was performed using a fixed batch size of 16, and model performance was evaluated at the end of each epoch on both training and validation sets. During validation, the model was set to evaluation mode.

A learning rate scheduler (Cosine Annealing) was used to adjust the learning rate dynamically across epochs to improve convergence stability.

4.3.2.4 Model Checkpointing

A checkpointing mechanism was used to retain the best-performing model based on validation accuracy. After each epoch, if the validation accuracy improved, the model weights were saved.

4.3.2.5 Evaluation Metrics

The model was evaluated using multiple performance metrics:

- Training and validation loss curves to analyze convergence behavior
- Training and validation accuracy curves to assess generalization
- Confusion matrix for class-wise performance visualization
- Classification report including precision, recall, and F1-score

4.3.3 ConvNeXt-Tiny Training Pipeline

The ConvNeXt-Tiny model was trained under the same experimental setup as EfficientNet-B0 to ensure a fair and controlled comparison. This includes identical dataset splits, batch size, preprocessing pipeline, loss function, optimizer (AdamW), and learning rate scheduler (Cosine Annealing).

4.3.3.1 Training Procedure

The training procedure followed the same strategy as EfficientNet-B0 for 30 epochs using mini-batch learning. The only architectural difference lies in the ConvNeXt-Tiny backbone, which provides improved feature representation compared to EfficientNet-B0.

During training, forward and backward propagation were performed using Cross-Entropy Loss, and model parameters were optimized using the AdamW optimizer. Validation was carried out at the end of each epoch in evaluation mode.

4.3.3.2 Model Checkpointing

A Cosine Annealing learning rate scheduler was used to improve convergence. The best-performing model was saved based on validation accuracy.

4.3.3.3 Evaluation

The same evaluation metrics as EfficientNet-B0 were used for consistency, including:

- Training and validation accuracy and loss curves
- Confusion matrix
- Classification report (precision, recall, F1-score)

4.3.4 Model Selection

Two deep learning architectures, EfficientNet-B0 and ConvNeXt-Tiny, were evaluated for the skin disease classification task under identical training conditions, including the same dataset split, preprocessing pipeline, loss function, optimizer, and learning rate scheduler. This ensured a fair and unbiased comparison between the two models.

EfficientNet-B0 was used as the baseline architecture which achieved a test accuracy of 77.84%, demonstrating reasonable performance on the dataset but showing limitations in capturing more complex visual patterns.

ConvNeXt-Tiny significantly outperformed the baseline, achieving a test accuracy of 84.62%. The improvement can be attributed to its modern architectural design.

Based on these results, ConvNeXt-Tiny was selected as the final model for the skin disease classification module due to its superior accuracy and better generalization performance.

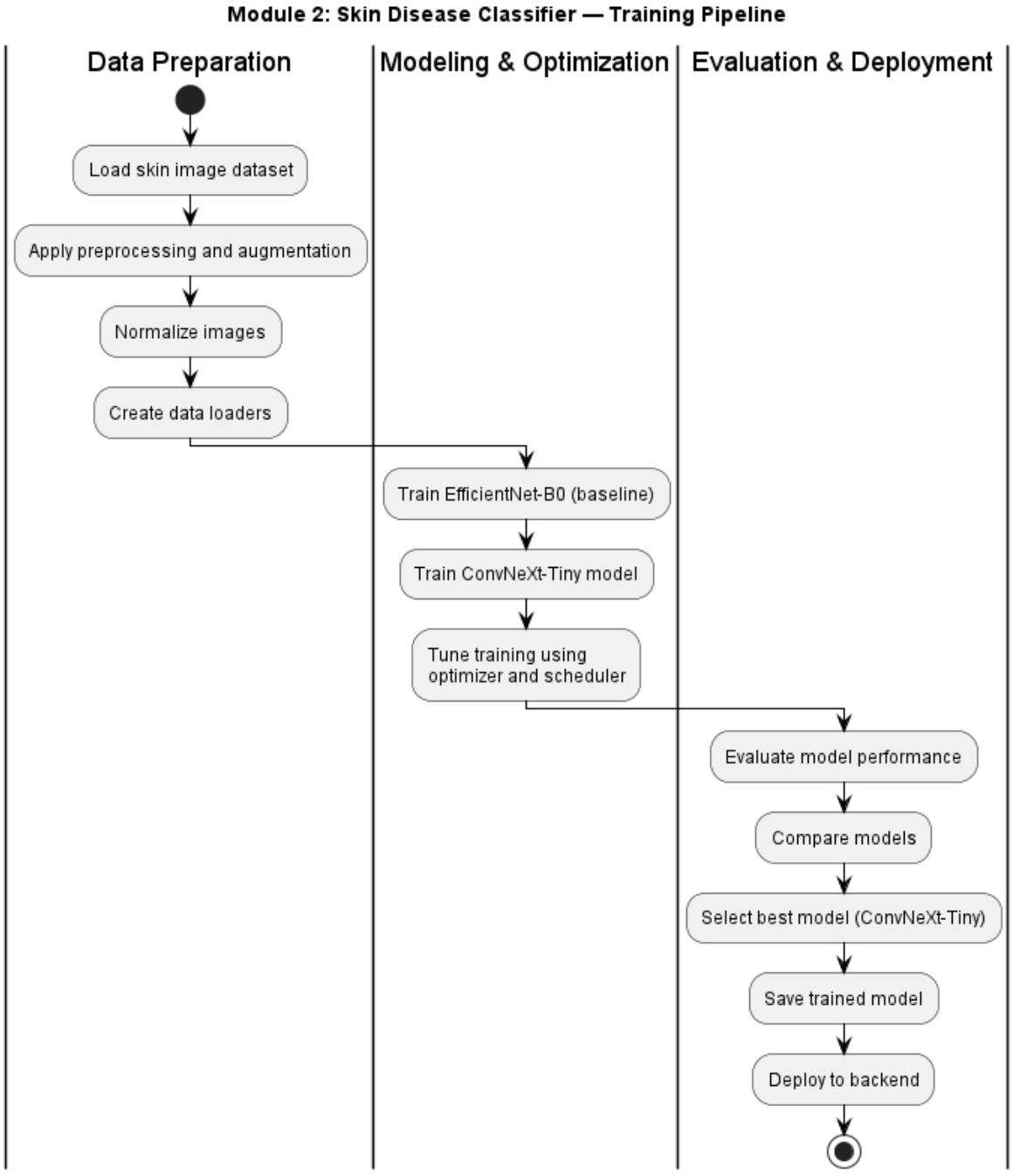

Figure 4.5: Module 2: Skin Disease Classifier - Training Pipeline

5 System Design

This chapter presents detailed diagrams and specifications for the MediAssist system architecture and components.

5.1 Overall System Architecture

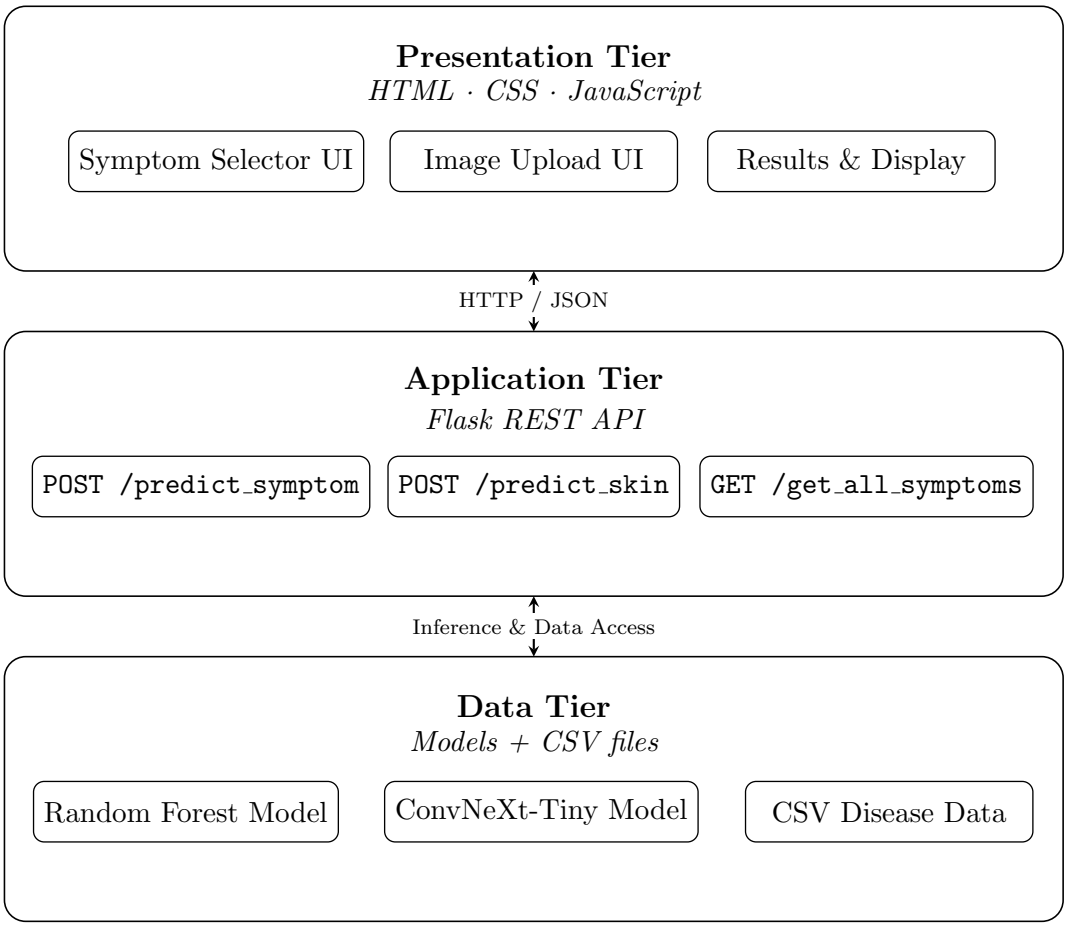

Figure 5.1: MediAssist Layered System Architecture

The system follows three-tier architecture:

Presentation Tier (Frontend): Web-based user interface built with HTML, CSS, JavaScript. Handles user interactions, input validation, and results display.

Application Tier (Backend): Flask-based REST API serving as the intermediary between frontend and models. Manages request routing, model invocation, and response formatting.

Data Tier: Consists of trained ML models (Random Forest, ConvNeXt-Tiny) and Disease information CSV files.

5.2 Module-Specific Design

5.2.1 Module 1: Symptom-Based Disease Predictor

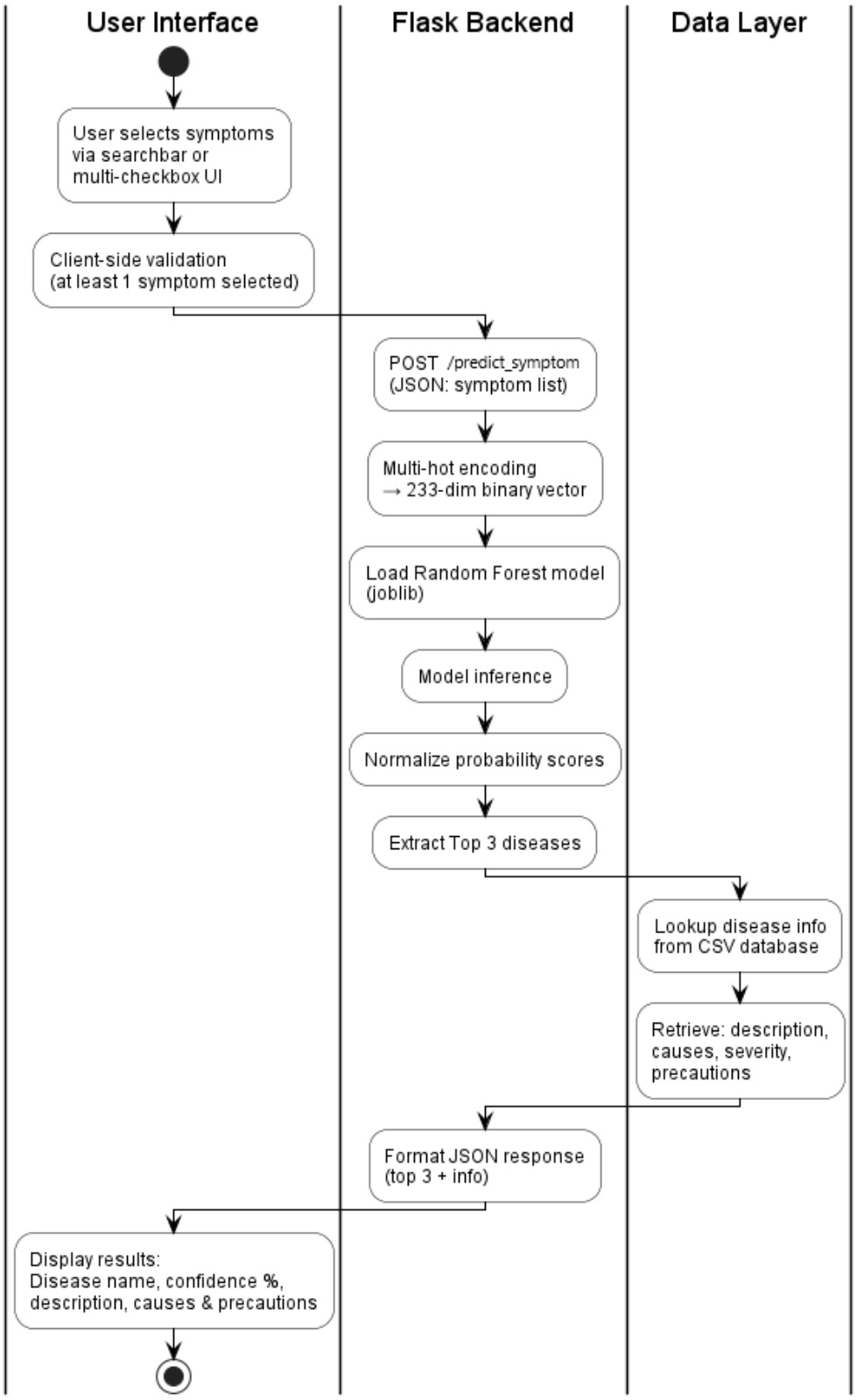

Figure 5.2: Symptom-Based Disease Predictor Operational Workflow

The operation pipeline for the symptom-based disease predictor proceeds as follows:

1. **User Interface:** The user selects symptoms via a searchbar or multi-checkbox UI, followed by client-side validation to ensure at least one symptom is selected before submission.

2. **API Request:** The selected symptoms are sent to the Flask backend via a `POST /predict_symptom` request as a JSON payload.

3. **Feature Encoding:** The symptom list is encoded into a 233-dimensional binary (multi-hot) vector, where each position represents the presence or absence of a specific symptom.

4. **Model Inference:** The encoded vector is passed to the pre-loaded Random Forest model (`joblib`), which generates raw class probabilities.

5. **Post-processing:** Probability scores are normalized and the top 3 predicted diseases are extracted.

6. **Data Lookup:** Disease information (description, causes, severity, and precautions) is retrieved from the CSV database for each of the top 3 predictions.

7. **Response & Display:** The backend formats a JSON response containing the predictions and disease information, which the frontend renders as structured results including disease info, and confidence percentage.

5.2.2 Module 2: Skin Disease Classifier

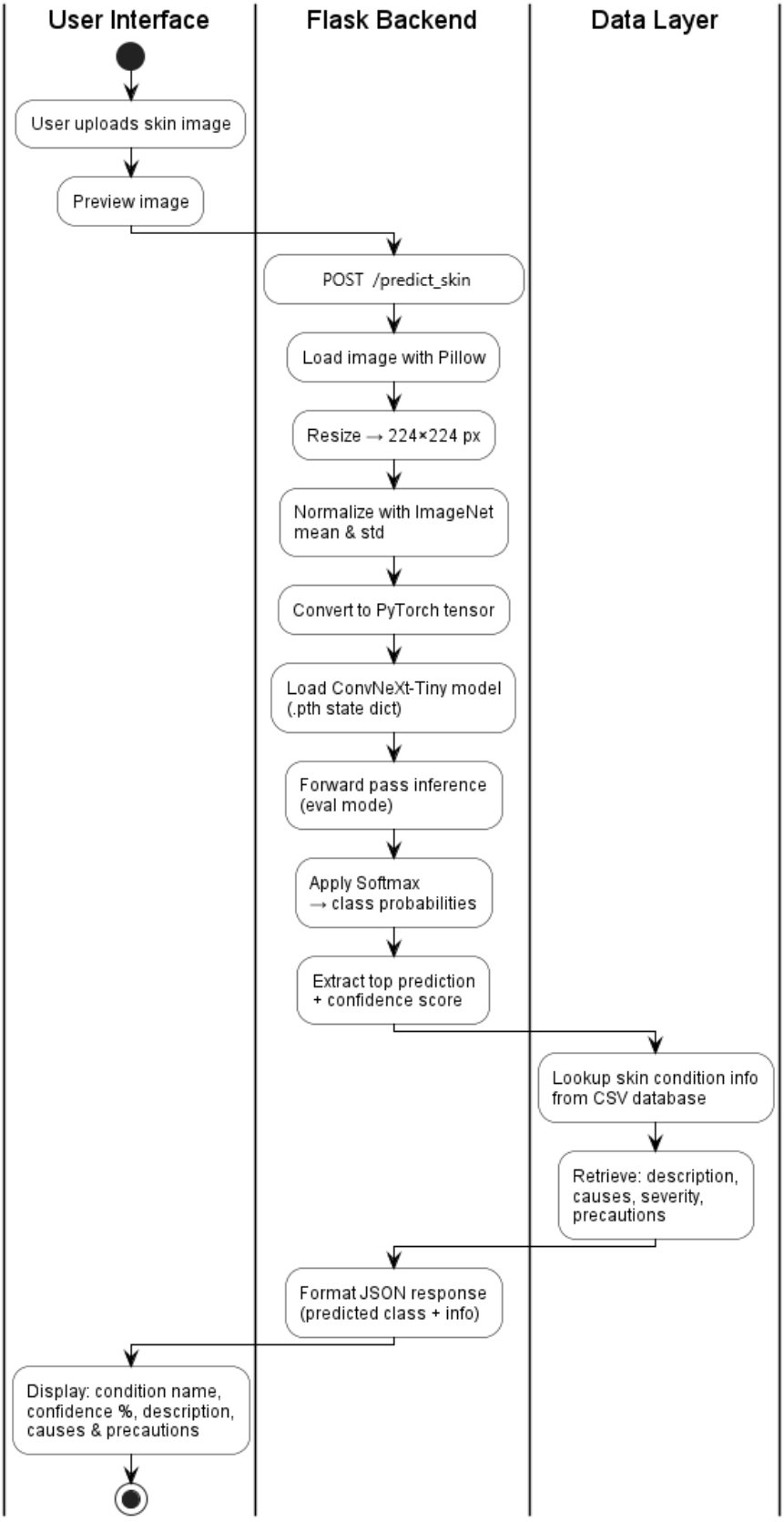

Figure 5.3: Skin Disease Classifier Operational Workflow

The operation pipeline for the skin disease classifier proceeds as follows:

1. **User Interface:** The user uploads a skin image, which is immediately previewed in the browser for verification before submission.

2. **API Request:** The image is transmitted to the Flask backend via a `POST /predict_skin` request.

3. **Image Preprocessing:** The backend loads the image using Pillow, resizes it to 224×224 pixels, normalizes it using ImageNet mean and standard deviation values, and converts it to a PyTorch tensor suitable for model input.

4. **Model Inference:** The preprocessed tensor is passed through the pre-loaded ConvNeXt-Tiny model (loaded from a `.pth` state dictionary) in evaluation mode to produce raw class logits.

5. **Post-processing:** A Softmax function is applied to the logits to obtain class probabilities, from which the top prediction and its confidence score are extracted.

6. **Data Lookup:** The predicted skin condition is used to retrieve corresponding information (description, causes, severity, and precautions) from the CSV database.

7. **Response & Display:** The backend formats a JSON response, and the frontend renders the condition name, confidence percentage, and associated health information.

6 Results & Discussion

This chapter presents the results obtained from implementing and evaluating MediAssist, followed by detailed discussion of findings and implications.

6.1 Results

6.1.1 Module 1: Symptom-Based Disease Predictor Results

The trained Random Forest model achieved strong performance on the test dataset, significantly outperforming the Decision Tree baseline.

Table 6.1: Module 1 Model Comparison: Decision Tree Baseline vs. Random Forest

Metric	**Decision Tree (Baseline)**	**Random Forest**
Test Accuracy	87.19%	96.68%
Best CV Accuracy (GridSearch)	–	97.05%
Macro-Averaged Precision	0.83	0.97
Macro-Averaged Recall	0.84	0.96
Macro-Averaged F1-Score	0.82	0.96
RF Best Hyperparameters: `n_estimators=200, max_depth=20, min_samples_leaf=4`		

The results demonstrate that the Random Forest model significantly outperforms the Decision Tree baseline across all evaluation metrics, confirming the effectiveness of ensemble learning for symptom-based disease prediction.

The test accuracy improved from 87.19% (Decision Tree) to 96.68% (Random Forest), representing an improvement of approximately 9.5 percentage points. This indicates that the Random Forest model generalizes much better to unseen patient data. Furthermore, the cross-validation accuracy of 97.05% is very close to the test accuracy, suggesting that the model is stable and not overfitting.

Significant improvements are also observed in macro-averaged metrics. Precision increased from 0.83 to 0.97, indicating fewer false positive predictions. Recall increased from 0.84 to 0.96, indicating better disease detection coverage. The F1-score increased from 0.82 to 0.96, showing a strong balance between precision and recall. These improvements are particularly important for multi-class medical classification, where performance across all disease categories must be considered equally.

The superior performance of Random Forest can be attributed to ensemble learning through multiple decision trees, which reduces variance and improves generalization.

The best hyperparameters obtained from GridSearch were `n_estimators = 200`, `max_depth = 20`, and `min_samples_leaf = 4`. Increasing the number of estimators improves stability, while controlling tree depth and enforcing a minimum number of samples per leaf helps reduce overfitting and produces smoother decision boundaries.

Overall, the Random Forest model provides good improvement over the Decision Tree baseline in terms of accuracy, robustness, and class-wise performance, making it the preferred model for symptom-based disease prediction.

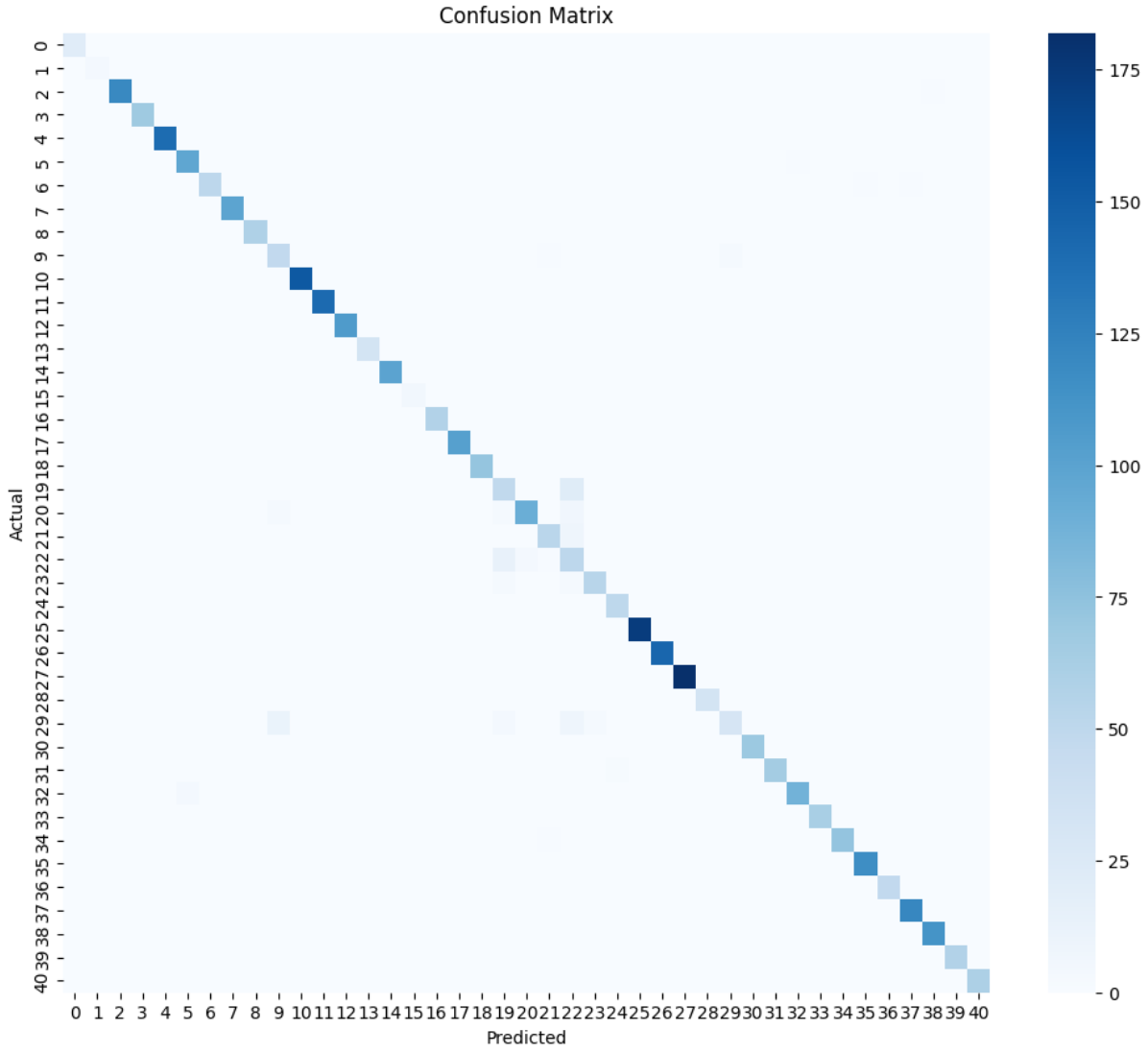

Figure 6.1: Confusion Matrix for Symptom-Based Disease Predictor (Random Forest)

6.1.2 Module 2: Skin Disease Classifier Results

The fine-tuned ConvNeXt-Tiny model demonstrated strong classification performance across 20 skin conditions, outperforming the EfficientNet-B0 baseline.

Table 6.2: Module 2 Model Comparison: EfficientNet-B0 vs. ConvNeXt-Tiny

Metric	**EfficientNet-B0**	**ConvNeXt-Tiny**
Overall Test Accuracy	77.84%	84.62%
Macro-Averaged Precision	0.74	0.82
Macro-Averaged Recall	0.76	0.82
Macro-Averaged F1-Score	0.75	0.82

The results show that ConvNeXt-Tiny outperforms EfficientNet-B0 across all evaluation metrics for skin disease classification.

The overall test accuracy increases from 77.84% (EfficientNet-B0) to 84.62% (ConvNeXt-Tiny), indicating a clear improvement in generalization performance. This suggests that ConvNeXt-Tiny is better at learning discriminative visual patterns from the skin disease dataset compared to EfficientNet-B0.

Similarly, all macro-averaged metrics show consistent gains. The macro-averaged precision improves from 0.74 to 0.82, recall increases from 0.76 to 0.82, and F1-score rises from 0.75 to 0.82 when moving from EfficientNet-B0 to ConvNeXt-Tiny. These improvements indicate that ConvNeXt-Tiny performs better not only in overall prediction accuracy but also in balancing performance across all classes.

The consistent increase in macro-averaged scores is particularly important in medical image classification, where class imbalance is common. It suggests that ConvNeXt-Tiny is more reliable in handling multiple skin disease categories without favoring dominant classes.

Overall, ConvNeXt-Tiny demonstrates superior and more balanced performance compared to EfficientNet-B0, making it the preferred model for the skin disease classification task.

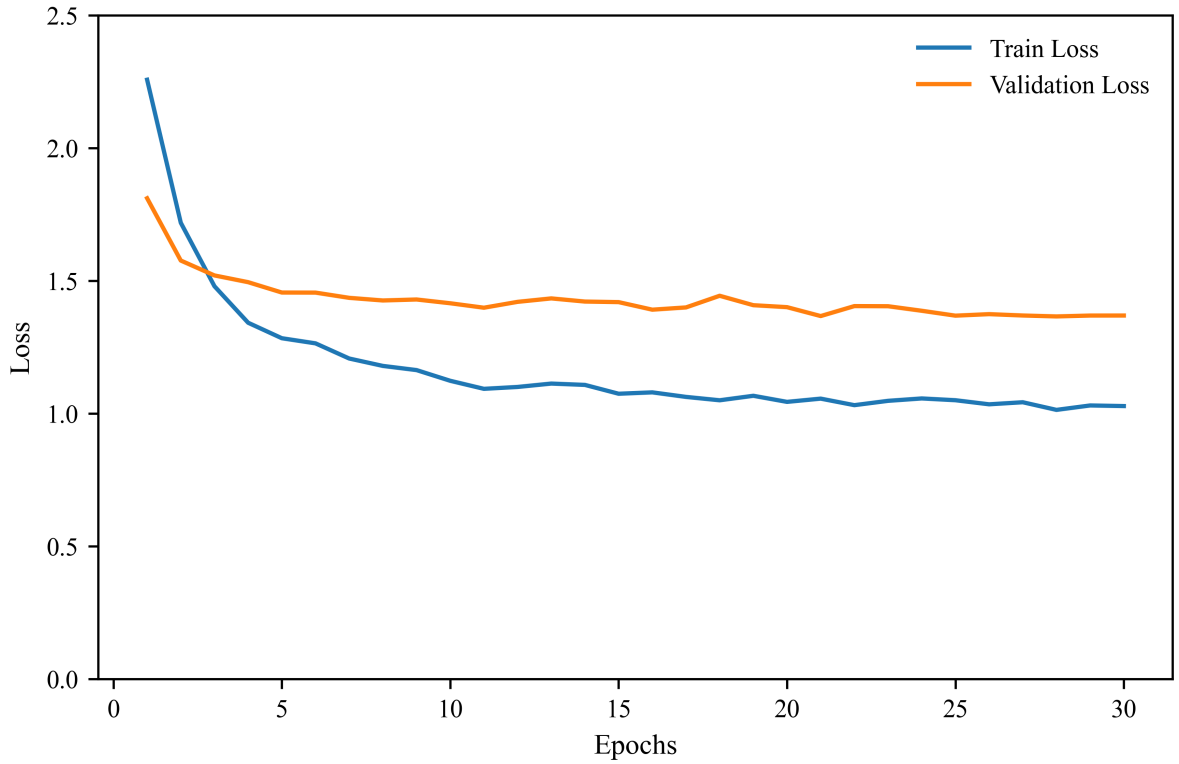

Figure 6.2: Training vs. Validation Loss Curve (ConvNeXt-Tiny)

The figure shows how the model's loss changed over 30 epochs of training. The training loss dropped sharply from approximately 2.3 at the beginning down to around 1.1 by epoch 10–12, then decreased more slowly to about 1 by epoch 30, showing continued improvement in fitting the training data.

The validation loss also decreased rapidly in the initial epochs, falling from around 1.8 to approximately 1.45 by epoch 5–7. After that, it fluctuated mildly but largely stabilized between 1.35–1.45 for the remainder of training. The smooth convergence of both training and validation loss curves to low values indicates that the model achieved good overall performance with stable and reliable results.

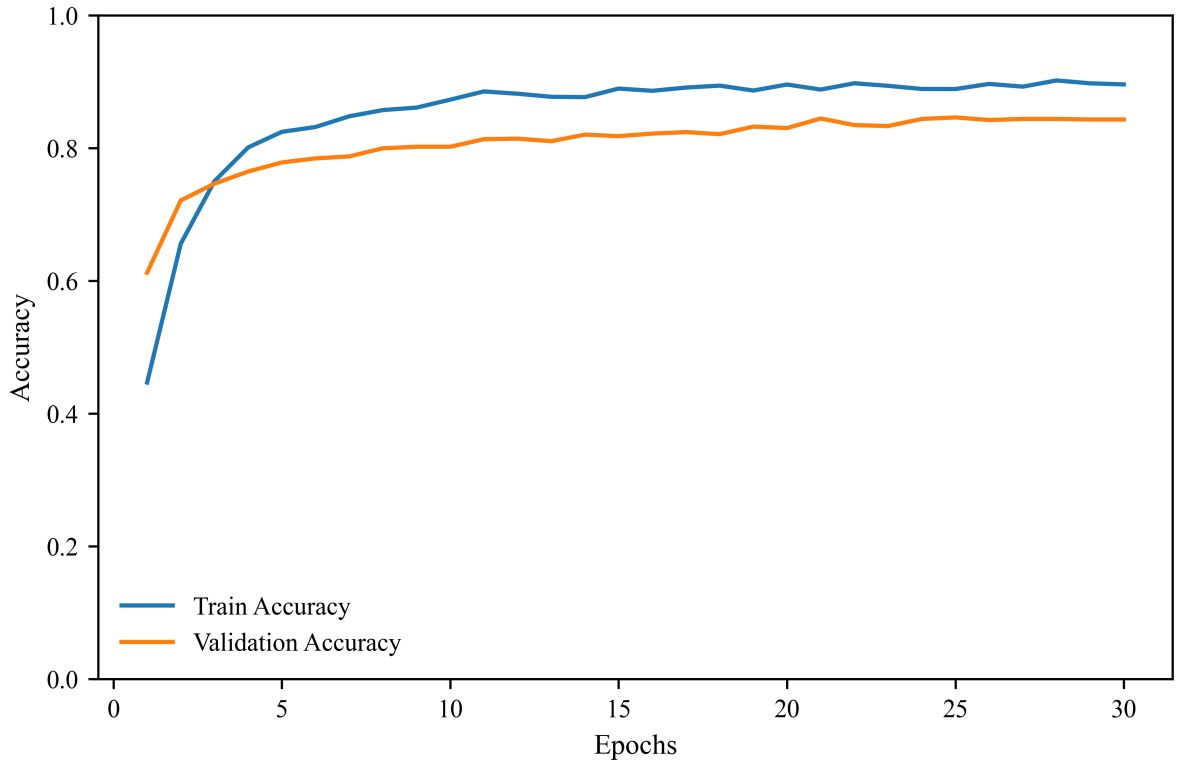

Figure 6.3: Training vs. Validation Accuracy Curve (ConvNeXt-Tiny)

The figure shows how the model's accuracy evolved over 30 epochs of training. The training accuracy increased rapidly from approximately 0.45 in the first epoch to around 0.82 by epoch 5, and continued to improve steadily, reaching approximately 0.9 by the end of training. This reflects strong learning and excellent performance on the training data.

The validation accuracy also rose quickly in the early epochs, climbing from around 0.61 to roughly 0.80 by epoch 5, and then stabilized at a high level between 0.83–0.85 for the remainder of training. Both training and validation accuracies achieved strong final values and remained stable in the later epochs, indicating that the model learned effectively and generalized well to unseen data.

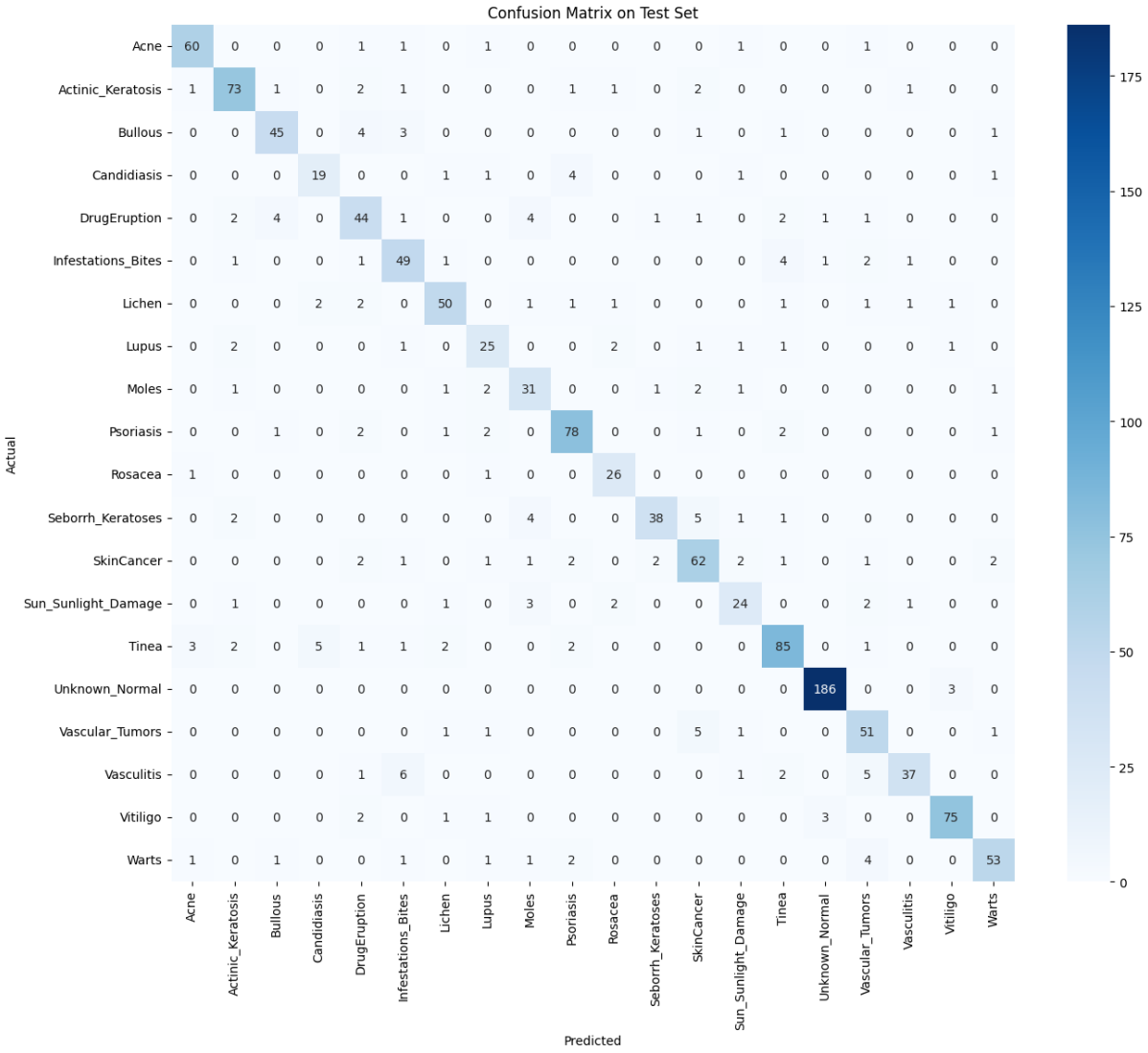

Figure 6.4: Confusion Matrix for Skin Disease Classifier (ConvNeXt-Tiny)

6.2 Discussion

6.2.1 Module 1: Symptom-Based Disease Prediction

The symptom-based disease prediction system demonstrates strong performance using both traditional machine learning approaches and ensemble learning techniques. The Decision Tree baseline achieves a test accuracy of 87.19%, indicating that even a single tree model can capture meaningful relationships between symptoms and diseases. However, its performance is limited by high variance and sensitivity to data splits.

In contrast, the Random Forest classifier significantly improves performance, achieving a test accuracy of 96.68%. This improvement is consistent across all evaluation metrics, including macro-averaged precision, recall, and F1-score, all of which increase to approximately 0.96. The close agreement between cross-validation accuracy (97.05%) and test accuracy further confirms that the model generalizes well and is not overfitting.

The improvement can be attributed to the ensemble nature of Random Forest, where multiple decision trees reduce variance and improve robustness. Hyperparameter tuning via GridSearchCV further enhances performance by selecting an optimal configuration (`n_estimators=200`, `max_depth=20`, `min_samples_leaf=4`), balancing model complexity and generalization ability.

Overall, the results indicate that ensemble learning is highly effective for structured symptom-based medical prediction tasks, where class boundaries are complex.

6.2.2 Module 2: Skin Disease Image Classification

For the image-based skin disease classification task, deep learning models were evaluated to determine the most effective architecture. EfficientNet-B0 is used as the baseline model, achieving a test accuracy of 77.84%. While it provides reasonable performance, it struggles to generalize across visually similar skin conditions.

ConvNeXt-Tiny significantly improves performance, achieving a test accuracy of 84.62%. This improvement is also reflected in macro-averaged precision, recall, and F1-score, all increasing from approximately 0.75 to 0.82. The consistent improvement across all metrics indicates better class-wise balance and reduced bias toward dominant classes.

The superior performance of ConvNeXt-Tiny can be attributed to its modern architecture, which incorporates design principles inspired by transformer models while retaining convolutional efficiency. This enables it to learn more robust visual representations compared to EfficientNet-B0.

6.2.3 Overall Discussion

Across both modules, the results demonstrate the effectiveness of modern machine learning and deep learning approaches for medical prediction tasks. In Module 1, ensemble learning significantly enhances performance over a single decision tree, while in Module 2, ConvNeXt-Tiny provides improved feature learning over EfficientNet-B0.

Overall, the system achieves strong predictive performance in both symptom-based and image-based disease classification, demonstrating the feasibility of a multi-modal AI-assisted diagnostic framework.

7 Conclusions

This project successfully developed *MediAssist*, an AI-based healthcare assistant that integrates symptom-based disease prediction and skin disease classification to provide accessible preliminary health guidance, particularly in resource-limited settings such as Nepal.

The symptom-based prediction module achieved a test accuracy of 96.68% across 41 diseases, while the skin disease classification module achieved 84.62% accuracy across 20 conditions, both achieving strong predictive performance for their respective tasks.

Finally, both modules were successfully integrated into a unified web application, providing efficient inference and stable performance. The system demonstrates the feasibility of combining traditional machine learning and deep learning approaches in a practical multi-modal AI-assisted diagnostic framework.

Key Achievements:

- Developed a dual-module healthcare system integrating symptom-based disease prediction and skin disease classification within a unified framework.
- Established a Decision Tree baseline for the symptom prediction module and demonstrated a 9.5 percentage point improvement in test accuracy using a Random Forest ensemble.
- Conducted comparative evaluation of deep learning architectures for skin disease classification, where ConvNeXt-Tiny outperformed EfficientNet-B0 by 6.78 percentage points.
- Implemented and deployed a fully functional web application enabling real-time inference for both modules.
- Designed an intuitive and user-friendly interface to improve accessibility and usability of the system.

Overall, MediAssist demonstrates the potential of AI to support early health awareness and improve accessibility, while not replacing professional medical care.

8 Limitations and Future Enhancement

8.1 Limitations

Despite achieving its objectives, MediAssist has several important limitations that should be acknowledged.

8.1.1 Dataset Limitations

The symptom predictor covers 41 common diseases in the training dataset, potentially missing rare or specialized conditions. The dataset of approximately 17,000 unique records, while larger than typical symptom datasets, may not reflect the full diversity of real clinical presentations. Class imbalance exists; class sizes range from 22 (acne) to 911 (hypothyroidism) and while class weights partially mitigate this, the lowest-accuracy classes (hepatitis variants, jaundice) remain challenging due to symptom overlap. The dataset lacks demographic information (age, gender) which could improve prediction accuracy in a clinical setting.

For the skin classifier, the training images, while sufficient for a student project, are limited compared to commercial dermatological datasets. Limited representation of different skin tones may affect performance across diverse populations. Image quality variations (lighting, blur, angle) can significantly impact classification accuracy but are not explicitly addressed in the current model.

8.1.2 Model Limitations

The Random Forest model, while effective, is a classical ML approach that may not capture complex symptom interaction patterns as well as modern deep learning methods could. The model cannot explain its predictions beyond feature importance, lacking the detailed reasoning a doctor would provide.

Although ConvNeXt-Tiny demonstrated strong performance in skin disease classification, the model's performance is dependent on the quality and diversity of the training dataset. The model is sensitive to variations in image quality, such as lighting conditions, blur, and background noise.

8.1.3 System Limitations

Performance depends on the user's ability to accurately identify and select symptoms or capture clear skin images. The web-only interface requires internet connectivity.

8.2 Future Enhancement

Several enhancements could significantly improve MediAssist's capabilities and impact.

8.2.1 Model Improvements

Expanding disease coverage to 100+ diseases through additional data collection and model retraining would make the system more comprehensive. Implementing deep learning approaches for symptom prediction could capture complex symptom interactions. Incorporating demographic information (age, gender, geographic region) could enable personalized predictions.

For the skin classifier, expanding to 30+ skin conditions through additional data collection would be valuable. Training on datasets with greater skin tone diversity would improve fairness and generalizability.

8.2.2 Feature Additions

Adding a conversational interface (chatbot) using Large Language Models could guide symptom elicitation more naturally.

8.2.3 Technical Enhancements

Creating native mobile applications for Android and iOS would improve accessibility and enable better camera integration for skin imaging. Implementing offline mode with local model inference would enable use in areas with limited internet connectivity, crucial for rural Nepal.

8.2.4 Clinical Validation

Running real-world tests where the system's predictions are compared with doctors' diagnoses to check how well it works. Also, working with hospitals in Nepal to test it on local patients so it fits the population better.

These enhancements would transform MediAssist from a student project into a comprehensive, validated, and impactful healthcare tool serving underserved populations.

References

[1] B. Kolukisa et al., "Diagnosis of coronary heart disease via classification algorithms," in *Proc. Int. Conf. Artificial Intelligence and Data Processing (IDAP)*, 2018, pp. 1-4.

[2] S. Uddin, A. Khan, M. E. Hossain, and M. A. Moni, "Comparing different supervised machine learning algorithms for disease prediction," *BMC Medical Informatics and Decision Making*, vol. 19, no. 1, pp. 1-16, 2019.

[3] M. Chen et al., "The application of machine learning techniques in clinical drug therapy decision support," in *Proc. IEEE Int. Conf. Bioinformatics and Biomedicine (BIBM)*, 2020, pp. 2342-2346.

[4] A. Sharma and R. Kumar, "Disease prediction using deep learning techniques: A comparative study," *Journal of Healthcare Engineering*, vol. 2021, 2021.

[5] A. Esteva et al., "Dermatologist-level classification of skin cancer with deep neural networks," *Nature*, vol. 542, no. 7639, pp. 115-118, 2017.

[6] H. A. Haenssle et al., "Man against machine: diagnostic performance of a deep learning convolutional neural network for dermoscopic melanoma recognition in comparison to 58 dermatologists," *Annals of Oncology*, vol. 29, no. 8, pp. 1836-1842, 2018.

[7] P. Tschandl, C. Rosendahl, and H. Kittler, "The HAM10000 dataset, a large collection of multi-source dermatoscopic images of common pigmented skin lesions," *Scientific Data*, vol. 5, no. 1, pp. 1-9, 2018.

[8] Z. Liu, H. Mao, C.-Y. Wu, C. Feichtenhofer, T. Darrell, and S. Xie, "A ConvNet for the 2020s," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2022, pp. 11976-11986.

[9] M. Tan and Q. V. Le, "EfficientNet: Rethinking model scaling for convolutional neural networks," in *Proc. Int. Conf. Machine Learning (ICML)*, 2019, pp. 6105-6114.

[10] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5-32, 2001.

[11] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278-2324, 1998.

[12] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet classification with deep convolutional neural networks," in *Advances in Neural Information Processing Systems*, 2012, pp. 1097-1105.

[13] S. J. Pan and Q. Yang, "A survey on transfer learning," *IEEE Transactions on Knowledge and Data Engineering*, vol. 22, no. 10, pp. 1345-1359, 2010.

Appendices

Appendix A: Source Code Repository

The complete source code for MediAssist is available on GitHub at:
`https://github.com/RimeshGithub/mediAssist`
The repository includes:

- Training notebooks for all models (Decision Tree & Random Forest, ConvNeXt-Tiny, EfficientNet-B0)
- Trained model files (.joblib for Random Forest, .pth for ConvNeXt-Tiny)
- Flask backend application code
- Frontend HTML/CSS/JavaScript files
- CSV files for disease info
- README with setup instructions

Appendix B: Dataset Sources

Module 1: Symptom-Disease Dataset

- Source: Kaggle – Disease Symptom Prediction Dataset (`itachi9604/disease-symptom-description-dataset`)
- License: CC0: Public Domain
- Size: 41 diseases, 233 symptoms

Module 2: Skin Disease Image Dataset

- Source: Kaggle – Skin Disease Dataset (`pacificrm/skindiseasedataset`)
- 20 skin condition classes
- Pre-split into train/test directories

Appendix C: User Interface Screenshots

Module 1: Symptom-Based Disease Predictor

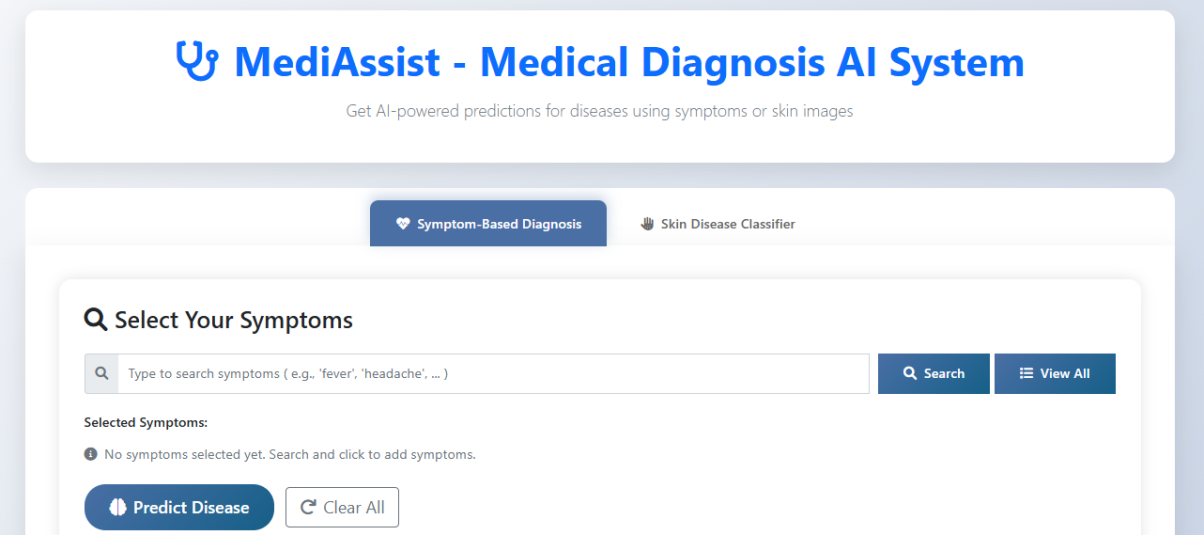

Figure C.1: UI for Symptom-Based Disease Predictor

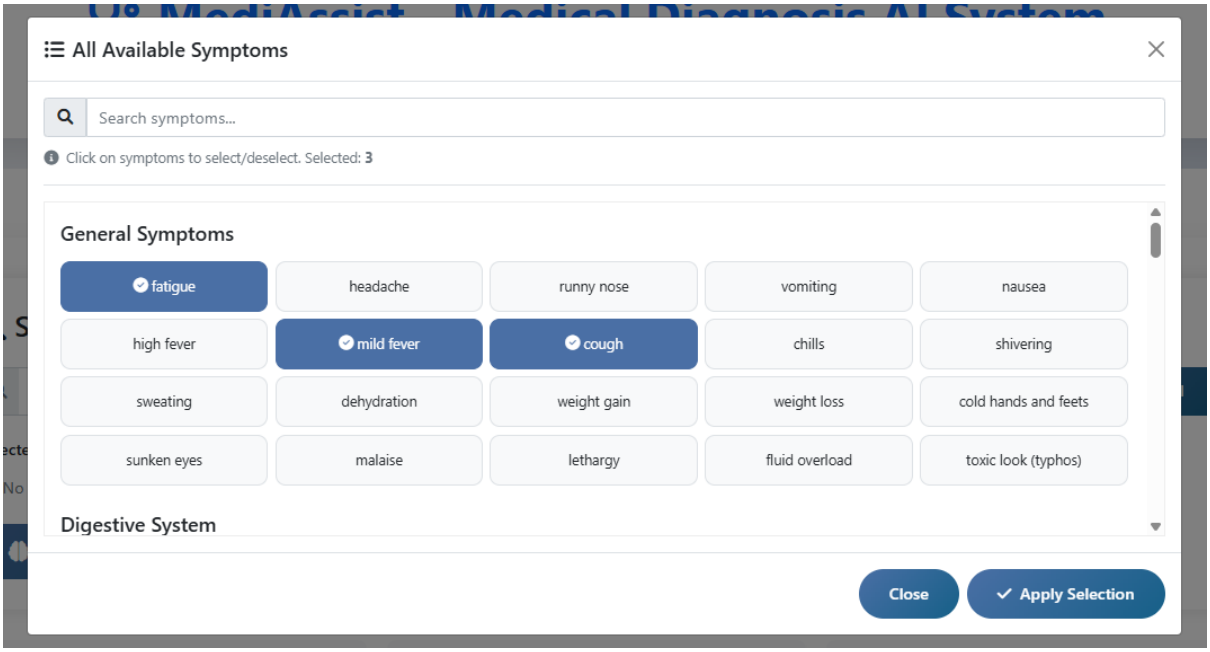

Figure C.2: Symptoms Selection

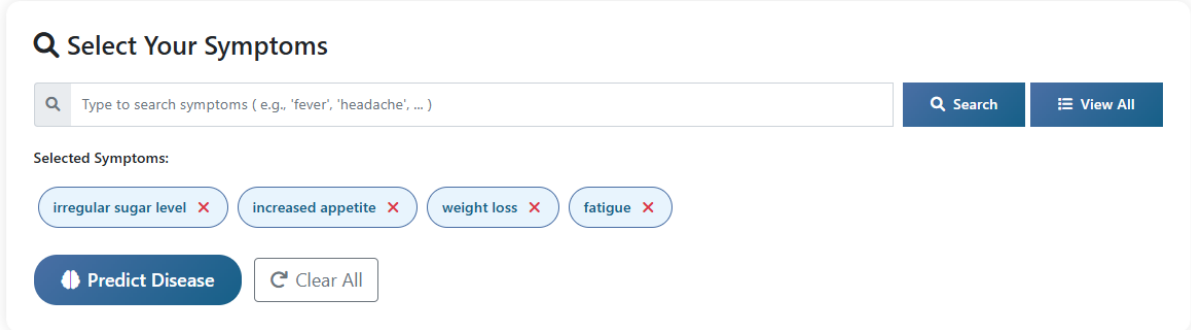

Figure C.3: Selected Symptoms

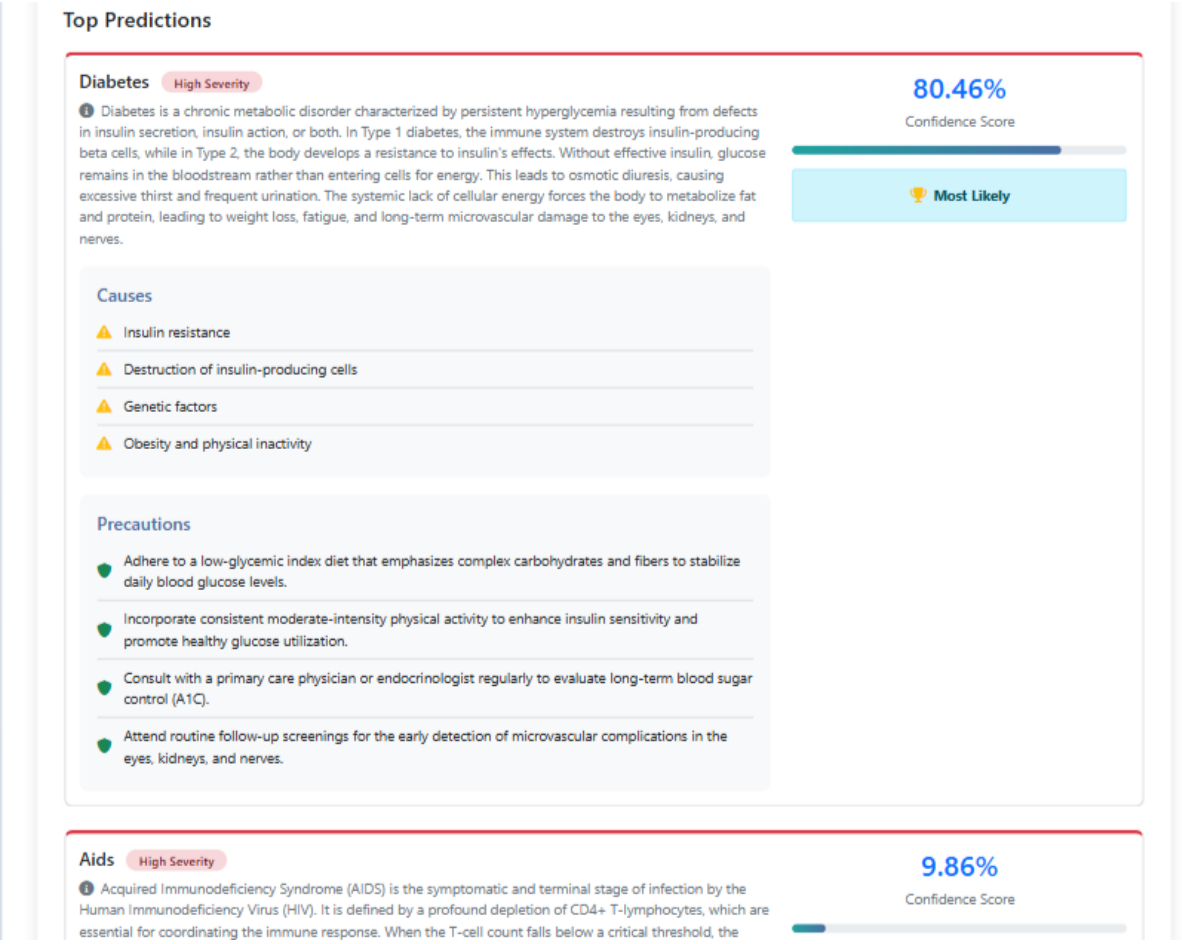

Figure C.4: Top Disease Predictions

Module 2: Skin Disease Classifier

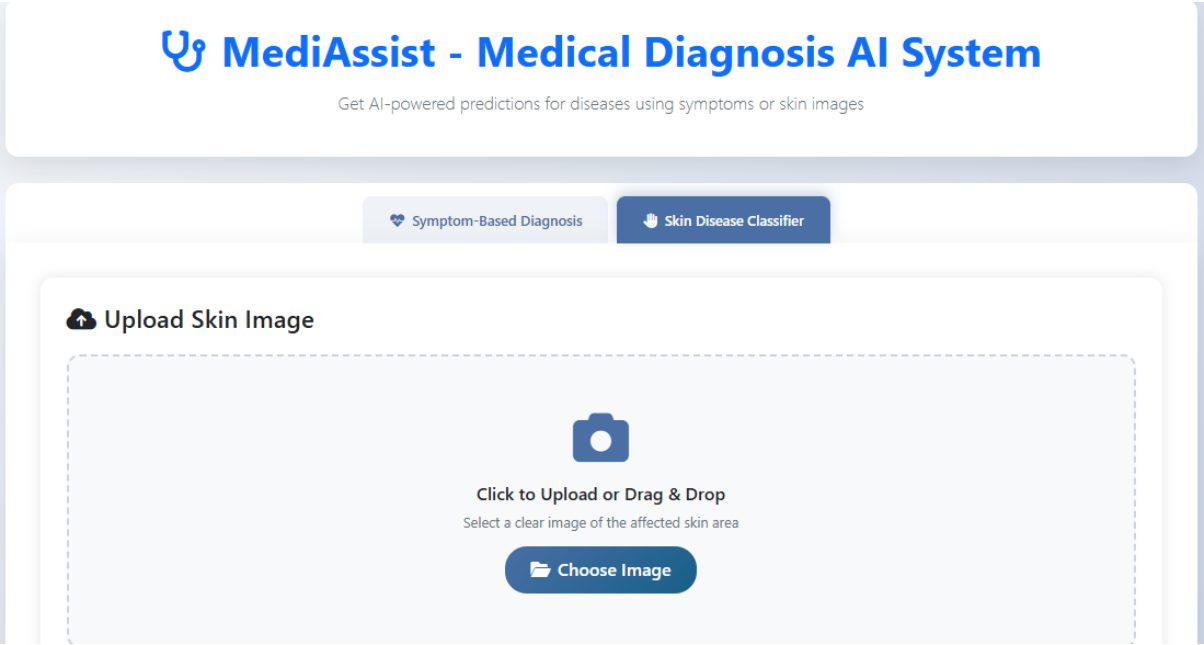

Figure C.5: UI for Skin Disease Classifier

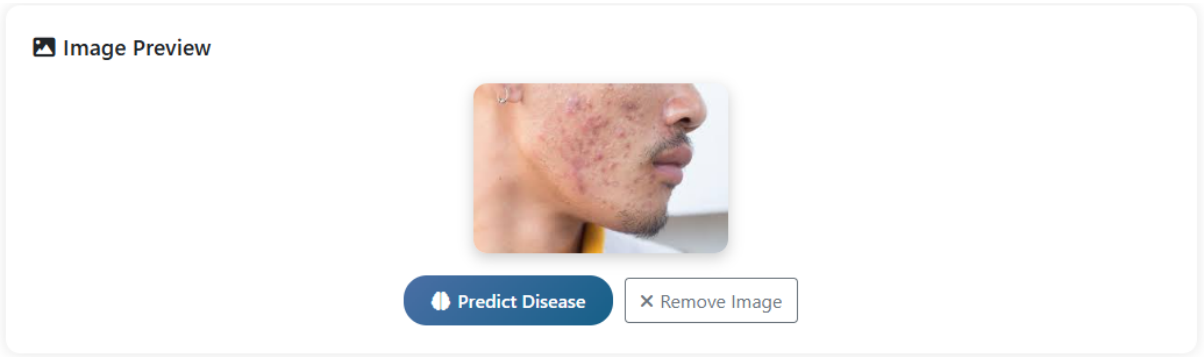

Figure C.6: Skin Image Selection

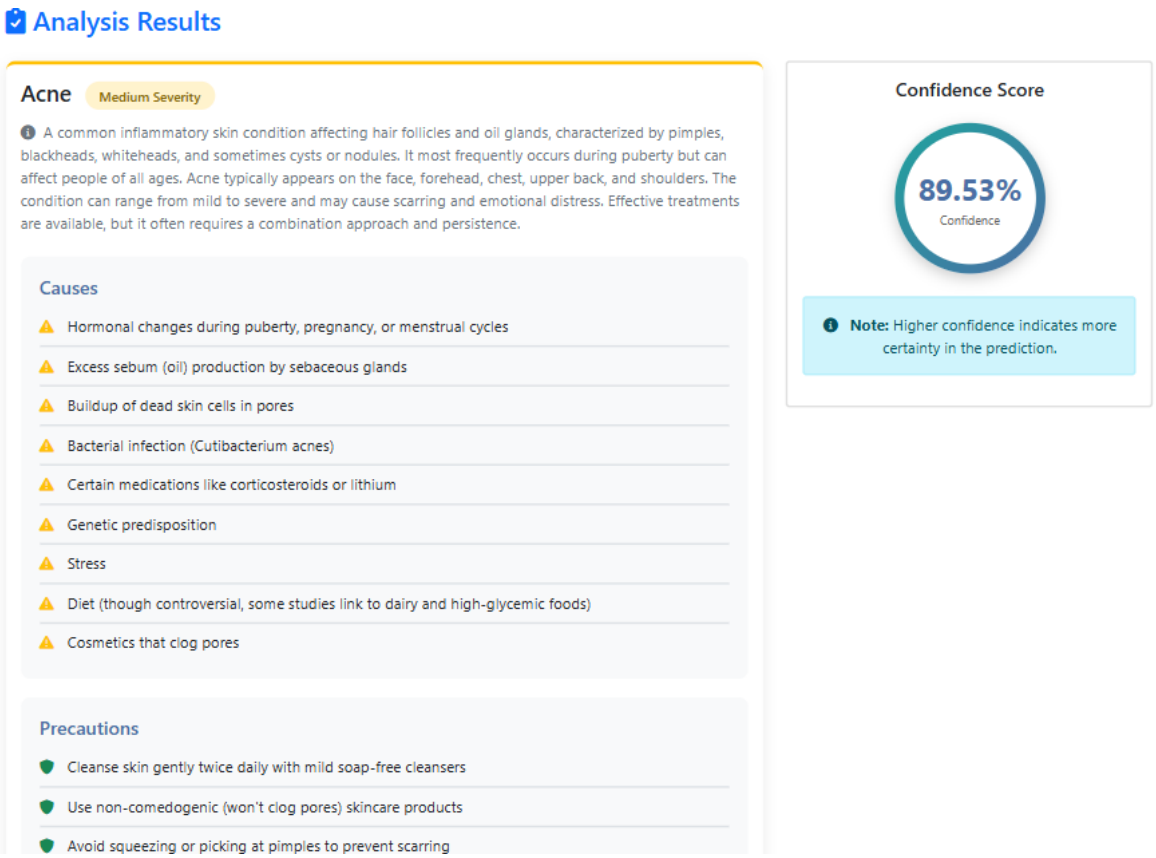

Figure C.7: Top Disease Prediction